GDMR Connect

API Documentation

Complete reference for all **277** endpoints of the GDMR Connect HRIS backend (Flask · MongoDB). Generated 15 October 2026 directly from the source code, so every route, permission check, rate limit, request field and response message shown here is what the code actually does.

Production base URL	https://api.gdmrconnect.com
Same API via the website	https://www.gdmrconnect.com/api/…
Authentication	Bearer token (JWT) from POST /api/login
Machine-readable version	docs/openapi.json (OpenAPI 3.0)
Regenerate	python tools/generate_api_docs.py

Contents

1. Overview 3

2. Endpoint index 6

3. Reference 7

- 3.1 System, Health & Files . . . 7
- 3.2 Authentication & Passwords . . . 7
- 3.3 Two-Step Verification (MFA) . . . 9
- 3.4 Special Access (Delegated Permissions) . . . 10
- 3.5 Employees, Managers, Departments . . . 11
- 3.6 Attendance — Check-in / Check-out . . . 17
- 3.7 Attendance Reports & Exports . . . 18
- 3.8 Biometric Devices . . . 19
- 3.9 Holidays & Attendance Calendar . . . 21
- 3.10 Leave Management . . . 22
- 3.11 Payroll . . . 24
- 3.12 Performance Management (PMS) . . . 26
- 3.13 PMS Compliance & Attendance Blocking . . . 30
- 3.14 Recruitment (ATS) . . . 32
- 3.15 Jobs & Employee Referrals . . . 36
- 3.16 Assessments . . . 38
- 3.17 Learning (LMS) . . . 40
- 3.18 Daily Work Plans . . . 43
- 3.19 Clients & Client Drive . . . 45
- 3.20 Team Chat . . . 47
- 3.21 AI Assistant . . . 51
- 3.22 Announcements, My Profile & Uploads . . . 51
- 3.23 Notifications . . . 54
- 3.24 Dashboard Statistics . . . 54
- 3.25 Assets . . . 55
- 3.26 Achievements . . . 57
- 3.27 My Goals (Sticky Notes) . . . 57
- 3.28 Experience Certificates (QR Verification) . . . 58

Appendix A — Public endpoints (no login) 61

Appendix B — Rate limits 61

Appendix C — Statistics 62

1. Overview

1.1 Conventions

Format. Requests and responses are JSON (`Content-Type: application/json`). File uploads use `multipart/form-data`; the reference marks those routes with “form fields” and “files”. Field names are `snake_case`.

IDs are MongoDB ObjectIds: 24-character hex strings (for example `66f1c2a9e4b0a1b2c3d4e5f6`). Path parts like `<job_id>` are such IDs unless the name says *token*.

Dates are `YYYY-MM-DD`; timestamps are ISO-8601 in UTC (the app displays them in IST). Booleans are real JSON booleans; money is a plain number in INR.

Lists are returned as JSON arrays (no pagination); a few endpoints accept `limit`/`days` style query parameters, shown per endpoint.

Compression and caching. JSON responses larger than 500 bytes are gzip-compressed when the client sends `Accept-Encoding: gzip`. Every `/api/…` response carries `Cache-Control: no-store`.

1.2 Authentication

1. **Log in:** `POST /api/login` with email and password. On success you get a token. Send it on every later request as `Authorization: Bearer <token>`.

2. **Token lifetime:** 4 hours, then the API answers `401 Session expired` and the user must log in again. There is no refresh token.

3. **Revocation:** each token carries a version number. Changing or resetting the password, “log out of all devices” (`POST /api/my/logout-all`), turning two-step verification off, or an admin force-logout invalidates every older token immediately.

4. **Two-step verification (optional):** if the account has it on, `POST /api/login` returns `{"mfa_required": true, "mfa_token": …}` instead of a session. Post that token and the 6-digit authenticator code (or a one-time backup code) to `POST /api/login/mfa` to receive the real session. The *mfa_token* lasts 5 minutes and cannot be used as a session.

5. **Lock-out:** 5 wrong passwords lock that email for 15 minutes (HTTP 423, identical for real and unknown emails). Wrong two-step codes are throttled the same way.

6. **Forgot password:** `POST /api/forgot-password` emails a single-use link (valid 30 minutes) to the web page `/reset-password/<token>`, which calls `POST /api/reset-password`. The password never changes until the link is used.

```
POST https://api.gdmrconnect.com/api/login
Content-Type: application/json

{"email": "name@company.com", "password": "••••••••"}

200 OK
{"token": "eyJhbGciOi…", "role": "employee", "password_changed": true,
 "user": {"_id": "66f1…", "name": "Asha K", "email": "name@company.com",
          "role": "employee", "department": "Brand", "photo_url": null}}

GET https://api.gdmrconnect.com/api/my/profile
Authorization: Bearer eyJhbGciOi…
```

Public routes (no login) are listed in Appendix A. Everything else answers `401` without a valid token.

1.3 Roles and permissions

Every user has one role. Rules are enforced inside each route, so the reference shows the exact check for each endpoint under **Access**.

Role	What it can do
owner	Everything an admin can do, plus owner-only notifications and approvals (for example a manager's own leave goes to the owner).
admin	All admin modules: employees, attendance, leave, payroll, PMS, recruitment, LMS, assessments, clients, assets, announcements and Special Access.
manager	Own data plus the department(s) they manage or head: team attendance and leave approvals, PMS reviews, team LMS, work by team, recruitment for their departments.

Role	What it can do
employee	Own data only: attendance, leave, payslips, PMS self-assessment, work plans, courses, referrals, chat, goals.

Special Access (delegated permissions). An admin or owner can give any employee temporary access to specific admin modules (`POST /api/admin/grant-access`). A grant has an access level — *view_only* or *view_edit* — a scope and an expiry. Routes marked “Special Access: “module”” accept anyone holding an active grant for that module; “(edit)” needs a *view_edit* grant. Grantable modules:

Module key	Feature
`employees`	Employees
`leaves`	Leave Requests
`attendance`	Attendance
`departments`	Departments
`manager`	Managers
`summary`	Reports
`pms`	PMS
`announcements`	Announcements
`assets`	Manage Assets
`work-by-team`	Work by Team
`clients`	Clients
`assessment`	Assessments
`lms`	LMS
`career`	Jobs
`ats`	Recruitment
`payroll`	Payroll
`certificates`	Experience Certificates

1.4 Errors

Errors are JSON with a human-readable message, for example `{"message": "Invalid ID"}`, and a meaningful HTTP status. Unexpected failures return a generic `500 {"message": "Something went wrong. Please try again."}` with no technical detail.

Status	Meaning
200	OK
201	Created
204	No content
400	Bad request
401	Not logged in / invalid or expired token
403	Not allowed
404	Not found
405	Method not allowed
409	Conflict
413	Request too large
423	Account temporarily locked
429	Too many requests
500	Server error
502	Upstream service failed
503	Not configured
504	Upstream timeout

1.5 Limits and protections

Item	Behaviour
Rate limits	Applied per route (Appendix B). Exceeding one returns 429. Counters are shared across workers only when REDIS_URL is configured.
Request size	JSON bodies over 2 MB are refused (413); no request may exceed 50 MB. Individual upload routes have tighter limits (typically 5–15 MB).
Uploads	Files are checked by their real contents: web pages, scripts and programs are refused and a file must match its extension (a .pdf must be a PDF). Stored on Cloudinary.
CORS	Browsers may call the API only from https://www.gdmrconnect.com, https://gdmrconnect.com and local development. Extra origins via the CORS_ORIGINS setting.
Security headers	Every response: X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy; HSTS over HTTPS; JSON responses also carry a restrictive Content-Security-Policy.
Links	Any URL a user can save (resume, recording, lesson links) must start with http:// or https://.

1.6 How the web app calls the API

The React app sends requests to `/api/…` on its own domain; Vercel forwards them to `https://api.gdmrconnect.com/api/…`. Each call has a 20-second timeout and up to 3 attempts (1 s and 2 s back-off) on network failures, then one XHR fallback (30 s) for networks that break HTTP/2. 401/403/423/429 are never retried. Team chat polls: conversations every 8 s, messages every 4 s, typing every 3 s, presence every 15–25 s, unread count every 20 s; admin badge counts every 60 s.

Scheduled jobs (IST) run inside the server, not through the API, except the end-of-day absence job which can be triggered with `POST /api/attendance/auto-absent` and the secret header `X-Cron-Secret`.

2. Endpoint index

Module	Endpoints	Public
System, Health & Files	4	4
Authentication & Passwords	6	3
Two-Step Verification (MFA)	6	1
Special Access (Delegated Permissions)	5	0
Employees, Managers, Departments	31	1
Attendance — Check-in / Check-out	6	0
Attendance Reports & Exports	6	0
Biometric Devices	11	3
Holidays & Attendance Calendar	6	0
Leave Management	8	0
Payroll	13	0
Performance Management (PMS)	22	0
PMS Compliance & Attendance Blocking	7	0
Recruitment (ATS)	17	2
Jobs & Employee Referrals	9	1
Assessments	10	3
Learning (LMS)	20	0
Daily Work Plans	10	0
Clients & Client Drive	12	0
Team Chat	19	0
AI Assistant	3	0
Announcements, My Profile & Uploads	16	0
Notifications	2	0
Dashboard Statistics	3	1
Assets	8	0
Achievements	3	0
My Goals (Sticky Notes)	6	0
Experience Certificates (QR Verification)	8	1

Total: 277 endpoint/method combinations — 257 require a login, 20 are public.

3. Reference

3.1 System, Health & Files

Service health checks and the legacy file-serving route.

Method	Path	Summary
GET	/	Home
GET	/api/health	Health check
GET	/health	Health
GET	/uploads/{path:filename}	Serve upload

GET `/`

Home

Access	Public (no login required)
Success	**200** plain text

GET `/api/health`

Health check

Access	Public (no login required)
Success	**200** JSON with status, db
Errors	**503** JSON with status, db, detail

GET `/health`

Health

Access	Public (no login required)
Success	**200** JSON with status

GET `/uploads/{path:filename}`

Serve upload

Access	Public (no login required)
Path	filename (file path)
Success	**200** file download

3.2 Authentication & Passwords

Login, password change, emailed password-reset links, log out of all devices and admin force-logout. Everything else in the API needs the token these routes issue.

Method	Path	Summary
POST	/api/admin/employees/{emp_id}/force-logout	Admin/owner ends every session of one person (e.g. a lost phone, a suspected account takeo
POST	/api/forgot-password	Forgot password
POST	/api/login	Login
POST	/api/my/logout-all	Logout all
POST	/api/my/set-password	Allows an authenticated user to update their password.
POST	/api/reset-password	Reset password

POST `/api/admin/employees/{emp_id}/force-logout`

Admin/owner ends every session of one person (e.g. a lost phone, a suspected account takeover).

Access	Logged in; Admin / Owner
Path	emp_id (ID)
Success	**200** All sessions for this account have been ended.
Errors	**400** Invalid ID **403** Unauthorized **404** Employee not found.

POST `/api/forgot-password`

Forgot password

Access	Public (no login) — part of signing in or recovering a password
Rate limit	5 per hour
JSON body	email

POST `/api/login`

Login

Access	Public (no login) — part of signing in or recovering a password
Rate limit	10 per minute
JSON body	password, email
Success	**200** JSON with mfa_required, mfa_token
Errors	**400** Email and password are required. **401** JSON **403** Your employment has ended. Portal access is no longer available. **423** Account locked due to too many failed attempts. Try again in … minute(s).

POST `/api/my/logout-all`

Logout all

Access	Any logged-in user (own records only)
Success	**200** You have been logged out of all devices.

POST `/api/my/set-password`

Allows an authenticated user to update their password.

Access	Any logged-in user (own records only)
JSON body	oldPassword, password
Success	**200** JSON with token — “Password updated successfully!”
Errors	**400** Current password is required. · Incorrect current password. Please try again. · Password must be at least 8 characters and contain 1 uppercase, 1 lowercase, 1 number, and 1 special character.

POST `/api/reset-password`

Reset password

Access	Public (no login) — part of signing in or recovering a password
Rate limit	10 per hour
JSON body	token, password
Success	**200** Your password has been changed. You can now log in.
Errors	**400** Password must be at least 8 characters and contain 1 uppercase, 1 lowercase, 1 number, and 1 special character.

3.3 Two-Step Verification (MFA)

Optional authenticator-app verification: second login step, set-up, backup codes, turn-off, and admin reset.

Method	Path	Summary
POST	/api/admin/employees/{emp_id}/reset-mfa	Reset mfa
POST	/api/login/mfa	Login mfa
POST	/api/my/mfa/disable	Disable
POST	/api/my/mfa/enable	Enable
POST	/api/my/mfa/setup	Setup
GET	/api/my/mfa/status	Status

POST /api/admin/employees/{emp_id}/reset-mfa

Reset mfa

Access	Logged in; Admin / Owner
Path	emp_id (ID)
Success	**200** Two-step verification was reset. They can set it up again after logging in.
Errors	**400** Invalid ID **403** Unauthorized **404** Employee not found.

POST /api/login/mfa

Login mfa

Access	Public (no login) — part of signing in or recovering a password
Rate limit	15 per minute
JSON body	mfa_token, code
Success	**200** JSON
Errors	**400** Code is required. **401** Your sign-in timed out. Please enter your password again.

POST /api/my/mfa/disable

Disable

Access	Any logged-in user (own records only)
Rate limit	10 per minute
JSON body	password, code
Success	**200** Two-step verification is off. Please log in again.
Errors	**400** Incorrect password. · That code is not correct. · Two-step verification is not on.

POST /api/my/mfa/enable

Enable

Access	Any logged-in user (own records only)
Rate limit	10 per minute
JSON body	code
Success	**200** JSON with backup_codes — “Two-step verification is on.”
Errors	**400** Start the setup first. · That code is not correct. Check the time on your phone and try again.

POST /api/my/mfa/setup

Setup

Access	Any logged-in user (own records only)
Success	**200** JSON with secret, otpauth_uri, qr
Errors	**400** Two-step verification is already on.

GET `/api/my/mfa/status`

Status

Access	Any logged-in user (own records only)
Success	**200** JSON with enabled, backup_codes_left

3.4 Special Access (Delegated Permissions)

Admins/owners grant an employee temporary access to chosen admin modules. These routes create, list and revoke those grants.

Method	Path	Summary
GET	/api/admin/active-grants	Get active grants
POST	/api/admin/grant-access	Grant access
GET	/api/admin/grantable-modules	Grantable modules
DELETE	/api/admin/revoke-access/{grant_id}	Revoke access
GET	/api/my/delegated-access	My delegated access

GET `/api/admin/active-grants`

Get active grants

Access	Logged in; Admin / Owner
Success	**200** JSON
Errors	**403** Unauthorized

POST `/api/admin/grant-access`

Grant access

Access	Logged in; Admin / Owner
JSON body	employeeId, accessLevel, scope, customDate, expiry, customExpiryTime, modules, module
Success	**201** JSON with id — “Temporary access granted successfully”
Errors	**400** Employee ID is required · Invalid employee or employee is already an admin. · Select at least one feature to grant access to. **403** Unauthorized

GET `/api/admin/grantable-modules`

Grantable modules

Access	Logged in; Admin / Owner
Success	**200** JSON list
Errors	**403** Unauthorized

DELETE `/api/admin/revoke-access/{grant_id}`

Revoke access

Access	Logged in; Admin / Owner
Path	grant_id (ID)
Success	**200** Access revoked successfully
Errors	**403** Unauthorized **404** Access grant not found or already inactive.

GET `/api/my/delegated-access`

My delegated access

Access	Any logged-in user (own records only)
Success	**200** JSON

3.5 Employees, Managers, Departments

The employee register: create/edit/remove staff, managers and admins, departments and heads, resignation/offboarding, profile lock, extended leave settings and personal-file documents.

Method	Path	Summary
GET	/api/admin/admins	List admins
DELETE	/api/admin/admins/{admin_id}	Delete admin
GET	/api/admin/departments	List departments
POST	/api/admin/departments	Create department
PUT	/api/admin/departments/{dept_id}	Update department
DELETE	/api/admin/departments/{dept_id}	Delete department
PUT	/api/admin/departments/{dept_name}/work-types	Set dept work types
GET	/api/admin/employees	List employees
POST	/api/admin/employees	Add employee
PUT	/api/admin/employees/{emp_id}	Edit employee
DELETE	/api/admin/employees/{emp_id}	Delete employee
POST	/api/admin/employees/{emp_id}/documents	Upload employee document
DELETE	/api/admin/employees/{emp_id}/documents/{doc_id}	Delete employee document
PUT	/api/admin/employees/{emp_id}/documents/{doc_id}/replace	Upload a new file version for an existing document entry, keeping the previous version in
POST	/api/admin/employees/{emp_id}/extended-leave	Add extended leave
DELETE	/api/admin/employees/{emp_id}/extended-leave/{leave_id}	Delete extended leave
POST	/api/admin/employees/{emp_id}/lock	Lock employee
GET	/api/admin/employees/{emp_id}/lock-audit	Employee lock audit
PUT	/api/admin/employees/{emp_id}/promote	Promote to manager
PUT	/api/admin/employees/{emp_id}/resignation	Set resignation
DELETE	/api/admin/employees/{emp_id}/resignation	Clear resignation
GET	/api/admin/employees/{emp_id}/status	Get employee status
POST	/api/admin/employees/{emp_id}/unlock	Unlock employee
GET	/api/admin/managers	List managers
PUT	/api/admin/managers/{man_id}	Edit manager
DELETE	/api/admin/managers/{man_id}	Delete manager
PUT	/api/admin/managers/{man_id}/role	Update manager role
GET	/api/departments/{dept_name}/work-types	Get dept work types
GET	/api/manager/my-employees	My employees
POST	/api/register-admin	Create an admin account.
POST	/api/register-manager	Register manager

GET `/api/admin/admins`

List admins

Access	Logged in; Admin / Owner
Success	**200** JSON
Errors	**403** Unauthorized

DELETE `/api/admin/admins/{admin_id}`

Delete admin

Access	Logged in; Admin / Owner; role check: admin
Path	admin_id (ID)
Success	**200** Admin account deleted
Errors	**400** Invalid ID · User is not an admin **403** Unauthorized · You cannot delete your own admin account **404** Admin not found

GET `/api/admin/departments`

List departments

Access	Logged in; Admin / Owner; Special Access: “ats”, “clients”, “departments”, “employees”, “manager”
Success	**200** JSON
Errors	**403** Unauthorized

POST `/api/admin/departments`

Create department

Access	Logged in; Admin / Owner; Special Access: “departments” (edit)
JSON body	name, legacy_name, description, head_ids, head_id
Success	**201** JSON
Errors	**400** Department '…' already exists. · Department name is required. **403** Unauthorized

PUT `/api/admin/departments/{dept_id}`

Update department

Access	Logged in; Admin / Owner; Special Access: “departments” (edit)
Path	dept_id (ID)
JSON body	name, description, head_ids, head_id
Success	**200** JSON
Errors	**400** Department '…' already exists. · Department name cannot be empty. · Invalid department ID. **403** Unauthorized **404** Department not found.

DELETE `/api/admin/departments/{dept_id}`

Delete department

Access	Logged in; Admin / Owner
Path	dept_id (ID)
Success	**200** plain text; Department '…' deleted and its employees unassigned.
Errors	**403** Unauthorized **404** Department not found.

PUT `/api/admin/departments/{dept_name}/work-types`

Set dept work types

Access	Logged in; Admin / Owner; role check: manager; managers see only their own departments
Path	dept_name (ID)
JSON body	types
Success	**200** JSON with types
Errors	**400** types must be a list **403** Unauthorized · Unauthorized: not your department **404** Department not found

GET `/api/admin/employees`

List employees

Access	Logged in; Admin / Owner; Special Access: “ats”, “attendance”, “career”, “clients”, “departments”, “employees”, “lms”, “manager”, “payroll”, “pms”, “summary”; role check: manager / owner
Query	active_only
Success	**200** JSON
Errors	**403** Unauthorized access.

POST `/api/admin/employees`

Add employee

Access	Logged in; Admin / Owner; Special Access: “employees” (edit)
JSON body	name, email, department, position, manager_id, doj, confirmation_date, promotion_date, shift, phone, location
Success	**201** JSON with id
Errors	**400** error message (varies with the cause) · Name and Email are required. · User with this email already exists **403** Unauthorized access. **500** Could not allocate an Employee ID — please try again.

PUT `/api/admin/employees/{emp_id}`

Edit employee

Access	Logged in; Admin / Owner; Special Access: “employees” (edit)
Path	emp_id (ID)
JSON body	name, department, position, location, email, phone, manager_id, shift, doj, employee_code, confirmation_date, promotion_date, employment_type
Success	**200** Employee profile updated successfully.
Errors	**400** Invalid employment_type. Must be 'Permanent', 'Contract', 'Internship' or 'Consultant'. · Invalid shift value. Must be 'morning', 'night', or 'general'. **403** Unauthorized

DELETE `/api/admin/employees/{emp_id}`

Delete employee

Access	Logged in; Admin / Owner
Path	emp_id (ID)
Success	**200** Employee completely removed from system.
Errors	**403** Unauthorized

POST `/api/admin/employees/{emp_id}/documents`

Upload employee document

Access	Logged in; Admin / Owner; Special Access: “employees” (edit)
Path	emp_id (ID)
Form fields	expiry_date, name, type
Files	file (multipart/form-data)
Success	**201** JSON
Errors	**400** error message (varies with the cause) · File too large (max 15 MB). Compress the PDF and try again. · Invalid ID · name and file are required · type must be one of: … **403** Unauthorized **404** Employee not found

DELETE `/api/admin/employees/{emp_id}/documents/{doc_id}`

Delete employee document

Access	Logged in; Admin / Owner; Special Access: “employees” (edit)
Path	emp_id (ID), doc_id (ID)
Success	**200** Document removed.
Errors	**400** Invalid ID **403** Unauthorized

PUT `/api/admin/employees/{emp_id}/documents/{doc_id}/replace`

Upload a new file version for an existing document entry, keeping the previous version in that entry's history[] instead of losing it (the old delete-then-reupload flow had no version trail at all).

Access	Logged in; Admin / Owner; Special Access: “employees” (edit)
Path	emp_id (ID), doc_id (ID)
Form fields	expiry_date
Files	file (multipart/form-data)
Success	**200** JSON with document — “Document replaced.”
Errors	**400** error message (varies with the cause) · File too large (max 15 MB). Compress the PDF and try again. · Invalid ID · file is required **403** Unauthorized **404** Document not found

POST `/api/admin/employees/{emp_id}/extended-leave`

Add extended leave

Access	Logged in; Admin / Owner; Special Access: “employees” (edit)
Path	emp_id (ID)
JSON body	from_date, to_date, type, notes
Success	**201** JSON
Errors	**400** Invalid date format. Use YYYY-MM-DD. · to_date cannot be before from_date. · type, from_date, and to_date are required. **403** This employee's profile is locked — contact an Admin. · Unauthorized

DELETE `/api/admin/employees/{emp_id}/extended-leave/{leave_id}`

Delete extended leave

Access	Logged in; Admin / Owner; Special Access: “employees” (edit)
Path	emp_id (ID), leave_id (ID)
Success	**200** JSON
Errors	**400** Invalid leave ID. **403** This employee's profile is locked — contact an Admin. · Unauthorized

POST `/api/admin/employees/{emp_id}/lock`

Lock employee

Access	Logged in; Admin / Owner
Path	emp_id (ID)
JSON body	reason
Success	**200** …'s profile is now locked.
Errors	**400** Invalid ID **403** Unauthorized — only Admin/Owner can lock an employee's profile. **404** Employee not found

GET `/api/admin/employees/{emp_id}/lock-audit`

Employee lock audit

Access	Logged in; Admin / Owner
Path	emp_id (ID)
Success	**200** JSON
Errors	**403** Unauthorized

PUT `/api/admin/employees/{emp_id}/promote`

Promote to manager

Access	Logged in; Admin / Owner; Special Access: “manager” (edit); role check: manager
Path	emp_id (ID)
Success	**200** Successfully promoted … to Manager of the … department.
Errors	**400** User is already a manager. **403** This employee's profile is locked — contact an Admin. · Unauthorized. Only admins can promote employees. **404** Employee not found.

PUT `/api/admin/employees/{emp_id}/resignation`

Set resignation

Access	Logged in; Admin / Owner; Special Access: “employees” (edit)
Path	emp_id (ID)
JSON body	notice_date, last_working_day, reason
Success	**200** JSON
Errors	**400** error message (varies with the cause) **403** This employee's profile is locked — contact an Admin. · Unauthorized

DELETE `/api/admin/employees/{emp_id}/resignation`

Clear resignation

Access	Logged in; Admin / Owner; Special Access: “employees” (edit)
Path	emp_id (ID)
Success	**200** JSON
Errors	**403** This employee's profile is locked — contact an Admin. · Unauthorized

GET `/api/admin/employees/{emp_id}/status`

Get employee status

Access	Logged in; Admin / Owner; Special Access: “employees”
Path	emp_id (ID)
Success	**200** JSON with extended_leaves, resignation
Errors	**403** Unauthorized

POST `/api/admin/employees/{emp_id}/unlock`

Unlock employee

Access	Logged in; Admin / Owner
Path	emp_id (ID)
JSON body	reason
Success	**200** …'s profile is now unlocked.
Errors	**400** Invalid ID **403** Unauthorized — only Admin/Owner can unlock an employee's profile. **404** Employee not found

GET `/api/admin/managers`

List managers

Access	Logged in; Admin / Owner; Special Access: “employees”, “manager”
Success	**200** JSON
Errors	**403** Unauthorized

PUT `/api/admin/managers/{man_id}`

Edit manager

Access	Logged in; Admin / Owner; Special Access: “manager” (edit)
Path	man_id (ID)
JSON body	name, department, email
Success	**200** Manager profile updated successfully.
Errors	**403** Unauthorized

DELETE `/api/admin/managers/{man_id}`

Delete manager

Access	Logged in; Admin / Owner
Path	man_id (ID)
Success	**200** Manager removed. Subordinates must be reassigned.
Errors	**403** Unauthorized

PUT `/api/admin/managers/{man_id}/role`

Update manager role

Access	Logged in; Admin / Owner; Special Access: “manager” (edit); role check: manager / owner
Path	man_id (ID)
JSON body	role
Success	**200** Role updated to '…'
Errors	**400** Invalid ID · User is not a manager or owner · role must be 'manager' or 'owner' **403** Unauthorized **404** User not found

GET `/api/departments/{dept_name}/work-types`

Get dept work types

Access	Any logged-in user
Path	dept_name (ID)
Success	**200** JSON with types

GET `/api/manager/my-employees`

My employees

Access	Logged in; role check: manager
Success	**200** JSON
Errors	**403** Unauthorized

POST `/api/register-admin`

Create an admin account.

Access	Open only while no admin/owner exists (first-time setup); otherwise needs an admin/owner login
Success	**201** JSON with id — “Master Admin created successfully”
Errors	**400** Admin with this email already exists · Email and password are required. · Password must be at least 8 characters with uppercase, lowercase, number, and special character. · Please enter a valid email address. **403** Maximum of 3 admin accounts allowed. · Unauthorized

POST `/api/register-manager`

Register manager

Access	Logged in; Admin / Owner
JSON body	name, email, password, department
Success	**201** Manager created successfully!
Errors	**400** Email already registered in the system. · Name, Email, Password, and Department are required **403** Unauthorized access.

3.6 Attendance — Check-in / Check-out

Photo check-in and check-out and the caller's own attendance history, plus the admin attendance listing.

Method	Path	Summary
GET	/api/admin/attendance	Master attendance log across every employee — backs AdminAttendancePage.jsx's 'Complete Lo
GET	/api/admin/attendance/{emp_id}	Employee attendance
GET	/api/admin/reports/late-checkins	HR report: every late check-in for a month — status_indicator is stamped "Present (Late)"
POST	/api/attendance/checkin-photo	Checkin photo
POST	/api/attendance/checkout-photo	Checkout photo
GET	/api/my/attendance	My attendance

GET `/api/admin/attendance`

Master attendance log across every employee — backs AdminAttendancePage.jsx's 'Complete Logs' tab.

Access	Logged in; Admin / Owner; Special Access: “attendance”
Success	**200** JSON
Errors	**403** Unauthorized

GET `/api/admin/attendance/{emp_id}`

Employee attendance

Access	Logged in; Admin / Owner; Special Access: “attendance”
Path	emp_id (ID)
Success	**200** JSON
Errors	**403** Unauthorized **404** Employee not found

GET `/api/admin/reports/late-checkins`

HR report: every late check-in for a month — status_indicator is stamped "Present (Late)" at check-in time itself (see checkin_photo() above), across all three shifts, so this is a direct query rather

Access	Logged in; Admin / Owner; Special Access: “attendance”, “summary”
Query	month
Success	**200** JSON
Errors	**400** month required **403** Unauthorized

POST `/api/attendance/checkin-photo`

Checkin photo

Access	Logged in; role check: employee / manager
JSON body	image, location
Success	**200** error message (varies with the cause)
Errors	**400** No image data received from frontend. **403** JSON with blocked · Unauthorized · Your employment has ended. Attendance check-in is no longer available.

POST `/api/attendance/checkout-photo`

Checkout photo

Access	Logged in; role check: employee / manager
JSON body	image, location
Success	**200** error message (varies with the cause)
Errors	**400** No image data provided **403** JSON with blocked · Unauthorized · Your employment has ended. Attendance check-out is no longer available.

GET `/api/my/attendance`

My attendance

Access	Any logged-in user (own records only)
Success	**200** JSON

3.7 Attendance Reports & Exports

Monthly/yearly attendance reports as PDF, CSV and Excel, including the multi-sheet Master Tracker.

Method	Path	Summary
GET	`/api/admin/reports/master-tracker-csv`	Master tracker csv
GET	`/api/admin/reports/master-tracker-pdf`	Master tracker pdf
GET	`/api/admin/reports/master-tracker-xlsx`	Master tracker xlsx
GET	`/api/admin/reports/monthly-attendance-csv`	Monthly attendance csv
GET	`/api/admin/reports/monthly-attendance-pdf`	Monthly attendance pdf
GET	`/api/admin/reports/monthly-attendance-xlsx`	Monthly attendance xlsx

GET `/api/admin/reports/master-tracker-csv`

Master tracker csv

Access	Logged in; Admin / Owner or Special Access “attendance” or “summary”
Query	year
Success	**200** file download
Errors	**400** plain text **403** plain text

GET `/api/admin/reports/master-tracker-pdf`

Master tracker pdf

Access	Logged in; Admin / Owner or Special Access “attendance” or “summary”
Query	year
Success	**200** file download
Errors	**400** plain text **403** plain text

GET `/api/admin/reports/master-tracker-xlsx`

Master tracker xlsx

Access	Logged in; Admin / Owner or Special Access “attendance” or “summary”
Query	year
Success	**200** file download
Errors	**400** plain text **403** plain text

GET `/api/admin/reports/monthly-attendance-csv`

Monthly attendance csv

Access	Logged in; Admin / Owner or Special Access “attendance” or “summary”
Query	month, year
Success	**200** file download
Errors	**400** plain text **403** plain text

GET `/api/admin/reports/monthly-attendance-pdf`

Monthly attendance pdf

Access	Logged in; Admin / Owner or Special Access “attendance” or “summary”
Query	month, year
Success	**200** file download
Errors	**400** plain text **403** plain text

GET `/api/admin/reports/monthly-attendance-xlsx`

Monthly attendance xlsx

Access	Logged in; Admin / Owner or Special Access “attendance” or “summary”
Query	month, year
Success	**200** file download
Errors	**400** plain text **403** plain text

3.8 Biometric Devices

Admin management of fingerprint/face devices and the device-facing ADMS endpoints (/iclock/*) that devices call directly.

Method	Path	Summary
GET	/api/admin/biometric-devices	List biometric devices
POST	/api/admin/biometric-devices	Add biometric device
DELETE	/api/admin/biometric-devices/{device_id}	Delete biometric device
GET	/api/admin/biometric-devices/{device_id}/enrollments	List device enrollments
DELETE	/api/admin/biometric-devices/{device_id}/enrollments/{pin}	Unlink device enrollment
POST	/api/admin/biometric-devices/{device_id}/enrollments/{pin}/map	Links (device_id, pin) to an employee.
POST	/api/admin/biometric-devices/{device_id}/test	Test biometric device
GET	/api/admin/biometric-devices/enrollments/by-employee/{employee_id}	Every device+pin this employee is linked to, across every device — backs the "Add Biometri
GET	/iclock/cdata	Handshake + periodic heartbeat.
POST	/iclock/cdata	Punch logs (table=ATTLOG) and locally-enrolled users (table=OPERLOG).
GET	/iclock/getrequest	Device polling for remote commands.

GET `/api/admin/biometric-devices`

List biometric devices

Access	Logged in; Admin / Owner or Special Access “attendance”
Success	**200** JSON list
Errors	**403** Unauthorized

POST `/api/admin/biometric-devices`

Add biometric device

Access	Logged in; Admin / Owner or Special Access “attendance” — edit
JSON body	name, branch, model, serial_number
Success	**200** JSON with _id, setup — “Device registered. Enter the setup details below on the device itself (Comm → Cloud Server Settings).”
Errors	**400** Name and serial number are required. **403** Unauthorized **409** A device with this serial number is already registered.

DELETE `/api/admin/biometric-devices/{device_id}`

Delete biometric device

Access	Logged in; Admin / Owner or Special Access “attendance” — edit
Path	device_id (ID)
Success	**200** Device removed. Existing attendance history is unaffected.
Errors	**400** Invalid device id. **403** Unauthorized **404** Device not found.

GET `/api/admin/biometric-devices/{device_id}/enrollments`

List device enrollments

Access	Logged in; Admin / Owner or Special Access “attendance”
Path	device_id (ID)
Success	**200** JSON
Errors	**403** Unauthorized

DELETE `/api/admin/biometric-devices/{device_id}/enrollments/{pin}`

Unlink device enrollment

Access	Logged in; Admin / Owner or Special Access “attendance” — edit
Path	device_id (ID), pin (ID)
Success	**200** Unlinked.
Errors	**403** Unauthorized **404** Not found.

POST `/api/admin/biometric-devices/{device_id}/enrollments/{pin}/map`

Links (device_id, pin) to an employee.

Access	Logged in; Admin / Owner or Special Access “attendance” — edit
Path	device_id (ID), pin (ID)
JSON body	employee_id, biometric_type
Success	**200** Linked successfully.
Errors	**400** Invalid employee id. · employee_id is required. **403** Unauthorized **404** Device not found.

POST `/api/admin/biometric-devices/{device_id}/test`

Test biometric device

Access	Logged in; Admin / Owner or Special Access “attendance”
Path	device_id (ID)
Success	**200** JSON with online, last_seen_at; JSON with online, last_seen_at — “This device has never contacted GDMR Connect yet. Double-check the Cloud Server Address/Port saved on the device, and that it has network access.”
Errors	**400** Invalid device id. **403** Unauthorized **404** Device not found.

GET `/api/admin/biometric-devices/enrollments/by-employee/{employee_id}`

Every device+pin this employee is linked to, across every device — backs the "Add Biometrics" action on an employee's profile, so it can show what's already linked before offering to add another.

Access	Logged in; Admin / Owner or Special Access “attendance”
Path	employee_id (ID)
Success	**200** JSON list
Errors	**403** Unauthorized

GET `/iclock/cdata`

Handshake + periodic heartbeat.

Access	Public address, used by biometric devices — the device's registered serial number (?SN=) is the credential
Rate limit	120 per minute
Query	SN
Success	**200** plain text
Errors	**400** plain text **403** plain text

POST `/iclock/cdata`

Punch logs (table=ATTLOG) and locally-enrolled users (table=OPERLOG).

Access	Public address, used by biometric devices — the device's registered serial number (?SN=) is the credential
Rate limit	120 per minute
Query	SN, table
Success	**200** plain text
Errors	**403** plain text **413** plain text

GET `/iclock/getrequest`

Device polling for remote commands.

Access	Public address, used by biometric devices — the device's registered serial number (?SN=) is the credential
Success	**200** plain text

3.9 Holidays & Attendance Calendar

Company holidays, attendance corrections and the calendar views built from them.

Method	Path	Summary
`GET`	`/api/admin/attendance/calendar`	Attendance calendar
`POST`	`/api/admin/holidays`	Add holiday
`DELETE`	`/api/admin/holidays/{holiday_id}`	Delete holiday
`GET`	`/api/holidays`	List holidays
`GET`	`/api/manager/attendance/calendar`	Attendance calendar
`GET`	`/api/my/attendance/calendar`	My attendance calendar

GET `/api/admin/attendance/calendar`

Attendance calendar

Access	Logged in; Admin / Owner; Special Access: “attendance”
Query	employee_id, month
Success	**200** JSON
Errors	**400** employee_id and month are required **403** Unauthorized **404** Employee not found or invalid month

POST `/api/admin/holidays`

Add holiday

Access	Logged in; Admin / Owner or Special Access “attendance” — edit
JSON body	date, name
Success	**201** JSON
Errors	**400** A holiday is already set on this date. · Holiday name is required · Invalid date · date must be in YYYY-MM-DD format **403** Unauthorized

DELETE `/api/admin/holidays/{holiday_id}`

Delete holiday

Access	Logged in; Admin / Owner or Special Access “attendance” — edit
Path	holiday_id (ID)
Success	**200** Holiday deleted.
Errors	**400** Invalid holiday ID **403** Unauthorized **404** Holiday not found

GET `/api/holidays`

List holidays

Access	Any logged-in user
Success	**200** JSON

GET `/api/manager/attendance/calendar`

Attendance calendar

Access	Logged in; role check: manager; managers see only their own departments
Query	employee_id, month
Success	**200** JSON
Errors	**400** Invalid employee_id · Invalid month · employee_id and month are required **403** Employee not in your department · Unauthorized

GET `/api/my/attendance/calendar`

My attendance calendar

Access	Any logged-in user (own records only)
Query	month
Success	**200** JSON
Errors	**400** Invalid month · month required

3.10 Leave Management

Apply for leave (with attachment), approve/reject, revoke, comp-off credits and balances.

Method	Path	Summary
GET	/api/admin/leaves	View leaves
PUT	/api/admin/leaves/{leave_id}	Update leave
GET	/api/comp-off/balances	Comp off balances
POST	/api/comp-off/grant	Grant comp off
POST	/api/leaves	Apply leave
POST	/api/leaves/{leave_id}/revoke	Cancels a leave request.
GET	/api/my/comp-off	My comp off
GET	/api/my/leaves	My leaves

GET `/api/admin/leaves`

View leaves

Access	Logged in; Special Access: “attendance”, “leaves”, “summary”; role check: admin / manager / owner
Success	**200** JSON
Errors	**403** Unauthorized

PUT `/api/admin/leaves/{leave_id}`

Update leave

Access	Logged in; Special Access: “leaves”, “leaves” (edit); role check: admin / manager / owner
Path	leave_id (ID)
JSON body	status
Success	**200** Leave updated successfully
Errors	**400** Invalid status **403** Unauthorized · Unauthorized: Your delegated access is View Only. · Unauthorized: employee is not in your team **404** Leave not found

GET `/api/comp-off/balances`

Comp off balances

Access	Logged in; Special Access: “attendance”, “leaves”; role check: admin / manager / owner
Success	**200** JSON
Errors	**403** Unauthorized

POST `/api/comp-off/grant`

Grant comp off

Access	Logged in; Special Access: “leaves” (edit); role check: admin / manager / owner; Admin / Owner, Special Access “leaves” (edit), or the employee's own manager
JSON body	employee_ids, employee_id, days, reason
Success	**200** Granted … comp-off day(s) to ….
Errors	**400** Pick at least one employee and a positive number of days. · That grant looks too large — please split it into smaller ones. · days must be a number **403** Nothing granted — those employees aren't on your team. · Unauthorized

POST `/api/leaves`

Apply leave

Access	Logged in; role check: employee / manager
Form fields	from_date, to_date, type, period, reason, date, comp_off
Files	attachment (multipart/form-data)
Success	**201** JSON with id — “Applied”
Errors	**400** error message (varies with the cause) · End date cannot be before start date · Invalid date format. · Leave application for past dates is limited to 7 days. · Not enough comp-off balance — you have … day(s), this request needs …. · Start and End dates are required **403** Unauthorized **409** You already have a … leave request covering ……. **500** File upload failed

POST `/api/leaves/{leave_id}/revoke`

Cancels a leave request.

Access	Logged in; role check: admin / manager / owner
Path	leave_id (ID)
Success	**200** Leave revoked.
Errors	**400** Invalid leave ID · Leave is already …. **403** Unauthorized **404** Leave not found

GET `/api/my/comp-off`

My comp off

Access	Any logged-in user (own records only)
Success	**200** JSON with balance, entries

GET `/api/my/leaves`

My leaves

Access	Any logged-in user (own records only)
Success	**200** JSON

3.11 Payroll

Salary structures, monthly payslips, loans and advances, and payroll export.

Method	Path	Summary
GET	/api/admin/payroll/export	Export payroll
GET	/api/admin/payroll/loans	List payroll loans
POST	/api/admin/payroll/loans	Create payroll loan
PUT	/api/admin/payroll/loans/{loan_id}/status	Update payroll loan status
GET	/api/admin/payroll/lop-preview	Auto-fill preview for the Run Payroll screen: per employee, the LOP day count already comp
GET	/api/admin/payroll/payslips	List payslips
PUT	/api/admin/payroll/payslips/{payslip_id}/status	Update payslip status
POST	/api/admin/payroll/run	Run payroll
GET	/api/admin/payroll/salaries	List salaries
PUT	/api/admin/payroll/salaries/{employee_id}	Upsert salary
GET	/api/admin/payroll/salaries/{employee_id}/history	Get salary history
GET	/api/my/loans	My loans
GET	/api/my/payslips	My payslips

GET `/api/admin/payroll/export`

Export payroll

Access	Logged in; Admin / Owner, Special Access “payroll”, or anyone in the Accounts department
Query	month, year, format
Success	**200** file download
Errors	**400** Invalid month or year · format must be 'xlsx' or 'pdf' · month (1-12) and year are required **403** Unauthorized

GET `/api/admin/payroll/loans`

List payroll loans

Access	Logged in; Admin / Owner, Special Access “payroll”, or anyone in the Accounts department
Query	status
Success	**200** JSON
Errors	**403** Unauthorized

POST `/api/admin/payroll/loans`

Create payroll loan

Access	Logged in; Admin / Owner, Special Access “payroll”, or anyone in the Accounts department — edit
JSON body	repayment_months, amount, emi_per_month, disbursed_date, reason, notes, employee_id, type
Success	**201** JSON
Errors	**400** amount must be greater than 0 · employee_id is required · type must be 'loan' or 'advance' **403** Unauthorized **404** Employee not found

PUT `/api/admin/payroll/loans/{loan_id}/status`

Update payroll loan status

Access	Logged in; Admin / Owner, Special Access “payroll”, or anyone in the Accounts department — edit
Path	loan_id (ID)
JSON body	status
Success	**200** Loan marked …
Errors	**400** Invalid ID · status must be 'active' or 'closed' **403** Unauthorized **404** Loan not found

GET `/api/admin/payroll/lop-preview`

Auto-fill preview for the Run Payroll screen: per employee, the LOP day count already computed by the Stage 1 attendance calendar (routes/calendar.py's classify_attendance_day -> "absent" -> "lop") fo

Access	Logged in; Admin / Owner, Special Access “payroll”, or anyone in the Accounts department
Query	month, year
Success	**200** JSON
Errors	**400** Invalid month or year · month (1-12) and year are required **403** Unauthorized

GET `/api/admin/payroll/payslips`

List payslips

Access	Logged in; Admin / Owner, Special Access “payroll”, or anyone in the Accounts department
Query	month, year, employee_id
Success	**200** JSON
Errors	**400** Invalid month · Invalid year **403** Unauthorized

PUT `/api/admin/payroll/payslips/{payslip_id}/status`

Update payslip status

Access	Logged in; Admin / Owner, Special Access “payroll”, or anyone in the Accounts department — edit
Path	payslip_id (ID)
JSON body	status
Success	**200** Payslip marked …
Errors	**400** Invalid ID · status must be 'Pending' or 'Paid' **403** Unauthorized **404** Payslip not found

POST `/api/admin/payroll/run`

Run payroll

Access	Logged in; Admin / Owner, Special Access “payroll”, or anyone in the Accounts department — edit
JSON body	adjustments, month, year
Success	**200** JSON with period, created, skipped — “Payroll run complete for ….”
Errors	**400** Invalid month or year · month (1-12) and year are required **403** Unauthorized

GET `/api/admin/payroll/salaries`

List salaries

Access	Logged in; Admin / Owner, Special Access “payroll”, or anyone in the Accounts department
Success	**200** JSON
Errors	**403** Unauthorized

PUT `/api/admin/payroll/salaries/{employee_id}`

Upsert salary

Access	Logged in; Admin / Owner, Special Access “payroll”, or anyone in the Accounts department — edit
Path	employee_id (ID)
JSON body	employee_code, designation, grade_profile, days_worked, bank_detail, transaction_detail, transaction_date, effective_date, bonus, increment_amount, increment_type, increment_reason
Success	**200** JSON with salary — “Salary structure saved”
Errors	**400** Invalid effective_date. Use YYYY-MM-DD. · Invalid employee ID **403** Unauthorized **404** Employee not found

GET `/api/admin/payroll/salaries/{employee_id}/history`

Get salary history

Access	Logged in; Admin / Owner, Special Access “payroll”, or anyone in the Accounts department
Path	employee_id (ID)
Success	**200** JSON list
Errors	**400** Invalid employee ID **403** Unauthorized **404** Employee not found

GET `/api/my/loans`

My loans

Access	Any logged-in user (own records only)
Success	**200** JSON

GET `/api/my/payslips`

My payslips

Access	Any logged-in user (own records only)
Success	**200** JSON

3.12 Performance Management (PMS)

Review templates, self-assessments, manager reviews, performance notes and warnings, recurring assignments.

Method	Path	Summary
GET	/api/admin/export-pms	Export pms
GET	/api/admin/pms	Admin's PMS page: every review org-wide, full stop — admin/owner outranks the manager/empl
GET	/api/admin/pms-dashboard	Dashboard
POST	/api/admin/pms-template	Save pms template
DELETE	/api/admin/pms-template/{template_id}	Delete pms template
GET	/api/admin/pms-template/{template_id}/export	CSV export of every response ever submitted against this one saved template, across every
GET	/api/admin/pms-template/{template_id}/responses	Every response ever submitted against this template, across every monthly cycle it's had (
GET	/api/admin/pms-templates	Every PMS form the current user owns, newest first — backs the "All PMS" tab in PMSWorkspa
POST	/api/manager/finalize-pms	Finalize pms review
GET	/api/manager/pms	Fetches all pending and completed PMS reviews for a manager's team.
GET	/api/manager/pms-calibration	Calibration
DELETE	/api/manager/pms/{review_id}	Permanently removes a review (self-assessment + any manager scoring) — same authorization

Method	Path	Summary
POST	/api/manager/pms/{review_id}/share	Marks a manager-owned review as visible on Admin's PMS page.
POST	/api/manager/pms/{review_id}/share-with-manager	Marks an admin/owner-owned review as visible to the employee's department manager(s) via G
GET	/api/my/pms	My pms
GET	/api/pms-template	Get pms template
POST	/api/pms/acknowledge	Acknowledge pms review
POST	/api/pms/performance-notes	A manager logs a performance note or a formal warning against one of their team; admin/own
DELETE	/api/pms/performance-notes/{note_id}	Delete performance note
POST	/api/pms/submit	Submit pms review
GET	/api/pms/workforce-dashboard	Workforce dashboard
GET	/api/pms/workforce-dashboard/{employee_id}	Full review history + performance notes for one employee — the "detailed report" behind a

GET `/api/admin/export-pms`

Export pms

Access	Logged in; Special Access: “pms”; role check: admin / manager / owner
Query	month
Success	**200** file download
Errors	**403** Unauthorized

GET `/api/admin/pms`

Admin's PMS page: every review org-wide, full stop — admin/owner outranks the manager/employee "Share with Admin" privacy flag rather than being gated by it.

Access	Logged in; Special Access: “pms”; role check: admin / owner
Success	**200** JSON
Errors	**403** Unauthorized

GET `/api/admin/pms-dashboard`

Dashboard

Access	Logged in; Special Access: “pms”; role check: admin / manager / owner; managers see only their own departments
Query	month
Success	**200** JSON
Errors	**403** Unauthorized

POST `/api/admin/pms-template`

Save pms template

Access	Logged in; Special Access: “pms” (edit); role check: admin / manager / owner; managers see only their own departments
JSON body	assigned_to, template_id, department, recurring, sessions, cycle_name, due_date, assign_day_of_month, due_day_of_month
Success	**200** JSON with _id — “PMS Form Assigned to … employees successfully!”; JSON with _id — “PMS form updated.”
Errors	**400** Assign day and due day must be numbers between 1 and 28. · Invalid template id. · You must assign the template to at least one employee. **403** Unauthorized **404** PMS form not found.

DELETE `/api/admin/pms-template/{template_id}`

Delete pms template

Access	Logged in; Special Access: “pms” (edit); role check: admin / manager / owner
Path	template_id (ID)
Success	**200** PMS form deleted.
Errors	**400** Invalid template id. **403** Unauthorized **404** PMS form not found.

GET `/api/admin/pms-template/{template_id}/export`

CSV export of every response ever submitted against this one saved template, across every monthly cycle — a full history export, distinct from /api/admin/export-pms which is scoped to a single month across all templates.

Access	Logged in; Special Access: “pms”; role check: admin / manager / owner
Path	template_id (ID)
Success	**200** file download
Errors	**400** Invalid template id. **403** Unauthorized **404** PMS form not found.

GET `/api/admin/pms-template/{template_id}/responses`

Every response ever submitted against this template, across every monthly cycle it's had (not just the current one) — the saved template's full response history, for the "view past responses" / export needs.

Access	Logged in; Special Access: “pms”; role check: admin / manager / owner
Path	template_id (ID)
Success	**200** JSON with cycle_name, recurring, responses
Errors	**400** Invalid template id. **403** Unauthorized **404** PMS form not found.

GET `/api/admin/pms-templates`

Every PMS form the current user owns, newest first — backs the "All PMS" tab in PMSWorkspace (active + expired), with enough detail (full `sessions`, raw `assigned_to`) to re-open one in the builder for editing.

Access	Logged in; Special Access: “pms”; role check: admin / manager / owner
Success	**200** JSON
Errors	**403** Unauthorized

POST `/api/manager/finalize-pms`

Finalize pms review

Access	Logged in; Special Access: “pms” (edit); role check: admin / manager / owner
JSON body	review_id, overall_rating, manager_scores, manager_feedback, development_plan, manager_comments
Success	**200** PMS Evaluation Review Completed successfully!
Errors	**400** Invalid overall_rating. Allowed: … · Invalid review ID · Review ID missing **403** Unauthorized · Unauthorized: this employee isn't on your team **404** Review not found

GET `/api/manager/pms`

Fetches all pending and completed PMS reviews for a manager's team.

Access	Logged in; Special Access: “pms”; role check: admin / manager / owner
Success	**200** JSON
Errors	**403** Unauthorized

GET `/api/manager/pms-calibration`

Calibration

Access	Logged in; Special Access: “pms”; role check: admin / manager / owner
Query	month
Success	**200** JSON
Errors	**403** Unauthorized

DELETE `/api/manager/pms/{review_id}`

Permanently removes a review (self-assessment + any manager scoring) — same authorization as finalizing one: a manager may only delete a review from their own team, admin/owner can delete any.

Access	Logged in; Special Access: “pms” (edit); role check: admin / manager / owner
Path	review_id (ID)
Success	**200** Review deleted.
Errors	**400** Invalid review ID **403** Unauthorized · Unauthorized — this employee isn't on your team. **404** Review not found

POST `/api/manager/pms/{review_id}/share`

Marks a manager-owned review as visible on Admin's PMS page.

Access	Logged in; Special Access: “pms” (edit); role check: admin / manager / owner
Path	review_id (ID)
Success	**200** Shared with Admin.
Errors	**400** Invalid review ID **403** Unauthorized · Unauthorized — this employee isn't on your team. **404** Review not found

POST `/api/manager/pms/{review_id}/share-with-manager`

Marks an admin/owner-owned review as visible to the employee's department manager(s) via GET /api/manager/pms.

Access	Logged in; Special Access: “pms” (edit); role check: admin / owner
Path	review_id (ID)
Success	**200** Shared with Manager.
Errors	**400** Invalid review ID **403** Unauthorized **404** Review not found

GET `/api/my/pms`

My pms

Access	Any logged-in user (own records only)
Success	**200** JSON

GET `/api/pms-template`

Get pms template

Access	Any logged-in user
Success	**200** JSON with sessions — “No active evaluations assigned to you.”; JSON with sessions, already_submitted — “You've already submitted this evaluation — see PMS History below.”

POST `/api/pms/acknowledge`

Acknowledge pms review

Access	Any logged-in user
JSON body	review_id
Success	**200** Review acknowledged
Errors	**400** Invalid review ID · Review has not been finalised by your manager yet. · review_id is required **403** Unauthorized. You can only acknowledge your own review. **404** Review not found

POST `/api/pms/performance-notes`

A manager logs a performance note or a formal warning against one of their team; admin/owner can log one against anyone.

Access	Logged in; Special Access: “pms” (edit); role check: admin / manager / owner
JSON body	employee_id, type, text
Success	**201** JSON with _id — “Saved.”
Errors	**400** employee_id and text are required. · type must be 'note' or 'warning'. **403** Unauthorized · Unauthorized: this employee isn't on your team

DELETE `/api/pms/performance-notes/{note_id}`

Delete performance note

Access	Logged in; Admin / Owner
Path	note_id (ID)
Success	**200** Deleted.
Errors	**400** Invalid id. **403** Unauthorized **404** Not found.

POST `/api/pms/submit`

Submit pms review

Access	Logged in; role check: admin / manager / owner
JSON body	template_id, responses
Success	**201** Self Assessment Submitted for Manager Review.

GET `/api/pms/workforce-dashboard`

Workforce dashboard

Access	Logged in; Special Access: “pms”; role check: admin / manager / owner
Query	month, department
Success	**200** JSON list
Errors	**403** Unauthorized

GET `/api/pms/workforce-dashboard/{employee_id}`

Full review history + performance notes for one employee — the "detailed report" behind a Dashboard row.

Access	Logged in; Special Access: “pms”; role check: admin / manager / owner
Path	employee_id (ID)
Success	**200** JSON with employee_id, name, department, reviews, notes
Errors	**403** Unauthorized · Unauthorized: this employee isn't on your team **404** Employee not found

3.13 PMS Compliance & Attendance Blocking

Tracks whether managers finished their reviews and (when switched on) blocks their team's check-in until they do.

Method	Path	Summary
GET	/api/admin/pms-compliance	Get pms compliance
POST	/api/admin/pms-compliance/{manager_id}/extend	Extend pms compliance deadline
GET	/api/admin/pms-compliance/{manager_id}/team	FRD .10 — 'View the employees affected by the block' for one manager+month.
POST	/api/admin/pms-compliance/{manager_id}/unblock	Unblock pms compliance
GET	/api/admin/pms-compliance/audit	Get pms compliance audit
GET	/api/admin/pms-compliance/settings	Get pms compliance settings
PUT	/api/admin/pms-compliance/settings	Update pms compliance settings

GET `/api/admin/pms-compliance`

Get pms compliance

Access	Logged in; Admin / Owner or Special Access “pms”
Query	month
Success	**200** JSON with month, blocking_enabled, rows
Errors	**403** Unauthorized

POST `/api/admin/pms-compliance/{manager_id}/extend`

Extend pms compliance deadline

Access	Logged in; Admin / Owner or Special Access “pms” — edit
Path	manager_id (ID)
JSON body	until, month, reason
Success	**200** Deadline extended to ….
Errors	**400** Invalid manager ID · until (YYYY-MM-DD) is required **403** Unauthorized **404** Manager not found

GET `/api/admin/pms-compliance/{manager_id}/team`

FRD .10 — 'View the employees affected by the block' for one manager+month.

Access	Logged in; Admin / Owner or Special Access “pms”
Path	manager_id (ID)
Query	month
Success	**200** JSON with manager, month, roster
Errors	**400** Invalid manager ID **403** Unauthorized **404** Manager not found

POST `/api/admin/pms-compliance/{manager_id}/unblock`

Unblock pms compliance

Access	Logged in; Admin / Owner or Special Access “pms” — edit
Path	manager_id (ID)
JSON body	month, reason, until
Success	**200** Attendance check-in restored for this manager's team.
Errors	**400** Invalid manager ID **403** Unauthorized **404** Manager not found

GET `/api/admin/pms-compliance/audit`

Get pms compliance audit

Access	Logged in; Admin / Owner or Special Access “pms”
Query	manager_id, month, limit
Success	**200** JSON
Errors	**403** Unauthorized

GET `/api/admin/pms-compliance/settings`

Get pms compliance settings

Access	Logged in; Admin / Owner or Special Access “pms”
Success	**200** JSON
Errors	**403** Unauthorized

PUT `/api/admin/pms-compliance/settings`

Update pms compliance settings

Access	Logged in; Admin / Owner or Special Access “pms” — edit
JSON body	blocking_enabled, exempt_employee_ids, exempt_department_names, new_joiner_grace_days
Success	**200** JSON
Errors	**400** new_joiner_grace_days must be a number **403** Unauthorized

3.14 Recruitment (ATS)

Candidates through the hiring pipeline: create/edit, resume upload with auto-fill, status changes, documents, recordings, dashboard activity and the AI overview.

Method	Path	Summary
GET	/api/admin/ats/activity	Everything the dashboard's "what happened" half needs, in one call: today vs yesterday, a
POST	/api/admin/ats/ai-overview	No question -> a structured overview (headline / highlights / risks / actions).
GET	/api/admin/ats/candidates	List candidates
POST	/api/admin/ats/candidates	Create candidate
GET	/api/admin/ats/candidates/{candidate_id}	Get candidate
PUT	/api/admin/ats/candidates/{candidate_id}	Update candidate
DELETE	/api/admin/ats/candidates/{candidate_id}	Delete candidate
POST	/api/admin/ats/candidates/{candidate_id}/doc-request	Doc request
PUT	/api/admin/ats/candidates/{candidate_id}/document	Review document
POST	/api/admin/ats/candidates/{candidate_id}/portfolio	Add portfolio
POST	/api/admin/ats/candidates/{candidate_id}/recording	Add recording
POST	/api/admin/ats/candidates/{candidate_id}/send-status-email	Send status email
PUT	/api/admin/ats/candidates/{candidate_id}/status	Update status
POST	/api/admin/ats/candidates/upload	Store a resume and return suggestions for the Add Candidate form.
GET	/api/admin/ats/stats	Stats
GET	/api/ats/documents/{doc_token}	Get doc checklist
POST	/api/ats/documents/{doc_token}	Upload doc

GET `/api/admin/ats/activity`

Everything the dashboard's "what happened" half needs, in one call: today vs yesterday, a recent activity feed, a daily trend, the hiring funnel and a list of candidates that need attention.

Access	Logged in; Recruitment access (admin/owner, HR department, or Special Access “ats”)
Query	days
Success	**200** JSON
Errors	**403** Unauthorized

POST `/api/admin/ats/ai-overview`

No question -> a structured overview (headline / highlights / risks / actions).

Access	Logged in; Recruitment access (admin/owner, HR department, or Special Access “ats”); managers see only their own departments
Rate limit	10 per minute
JSON body	refresh, question
Success	**200** JSON with cached; JSON with mode, answer, as_of
Errors	**400** Question is too long (max 400 characters). **403** Unauthorized **502** Couldn't generate the overview right now. Please try again. **503** AI is not configured on the server. **504** The AI took too long to respond. Please try again.

GET `/api/admin/ats/candidates`

List candidates

Access	Logged in; Recruitment access (admin/owner, HR department, or Special Access “ats”); managers see only their own departments
Success	**200** JSON
Errors	**403** Unauthorized

POST `/api/admin/ats/candidates`

Create candidate

Access	Logged in; Recruitment access (admin/owner, HR department, or Special Access “ats”) — edit
JSON body	skills, experience, phone, department, job_role, source, campaign, education, current_company, current_ctc, expected_ctc, current_location, preferred_location, notice_period, resume_url, remarks, joining_date, employment_type, contract_months, name, email, sourced_by, referred_by
Success	**201** JSON
Errors	**400** A candidate with this email already exists. · A candidate with this phone number already exists. · Invalid referred_by employee ID · Invalid sourced_by employee ID · Please enter a valid 10-digit phone number. · Please enter a valid email address. · The resume link must start with http:// or https:// · name is required **403** Unauthorized

GET `/api/admin/ats/candidates/{candidate_id}`

Get candidate

Access	Logged in; Recruitment access (admin/owner, HR department, or Special Access “ats”); managers see only their own departments
Path	candidate_id (ID)
Success	**200** JSON
Errors	**400** Invalid ID **403** Access denied · Unauthorized **404** Candidate not found

PUT `/api/admin/ats/candidates/{candidate_id}`

Update candidate

Access	Logged in; Recruitment access (admin/owner, HR department, or Special Access “ats”) — edit
Path	candidate_id (ID)
JSON body	name, email, skills, experience, source, sourced_by, referred_by, phone, department, job_role, campaign, education, current_company, current_ctc, expected_ctc, current_location, preferred_location, notice_period, resume_url, remarks, joining_date, employment_type, contract_months
Success	**200** JSON
Errors	**400** A candidate with this email already exists. · A candidate with this phone number already exists. · Invalid ID · Invalid … employee ID · Please enter a valid 10-digit phone number. · Please enter a valid email address. · The resume link must start with http:// or https:// **403** Unauthorized **404** Candidate not found

DELETE `/api/admin/ats/candidates/{candidate_id}`

Delete candidate

Access	Logged in; Recruitment access (admin/owner, HR department, or Special Access “ats”) — edit
Path	candidate_id (ID)
Success	**200** deleted
Errors	**400** Invalid ID **403** Unauthorized **404** Candidate not found

POST `/api/admin/ats/candidates/{candidate_id}/doc-request`

Doc request

Access	Logged in; Recruitment access (admin/owner, HR department, or Special Access “ats”) — edit
Path	candidate_id (ID)
JSON body	required_docs
Success	**200** JSON with upload_link, token — “Document request sent”
Errors	**400** Invalid ID **403** Unauthorized **404** Candidate not found

PUT `/api/admin/ats/candidates/{candidate_id}/document`

Review document

Access	Logged in; Recruitment access (admin/owner, HR department, or Special Access “ats”) — edit
Path	candidate_id (ID)
JSON body	name, status
Success	**200** JSON with documents
Errors	**400** Invalid ID · name is required · status must be one of: … **403** Unauthorized **404** Candidate not found · Document '…' not found on this candidate

POST `/api/admin/ats/candidates/{candidate_id}/portfolio`

Add portfolio

Access	Logged in; Recruitment access (admin/owner, HR department, or Special Access “ats”) — edit
Path	candidate_id (ID)
JSON body	url
Success	**200** Portfolio link added
Errors	**400** Invalid ID · The link must start with http:// or https:// · url is required **403** Unauthorized **404** Candidate not found

POST `/api/admin/ats/candidates/{candidate_id}/recording`

Add recording

Access	Logged in; Recruitment access (admin/owner, HR department, or Special Access “ats”) — edit
Path	candidate_id (ID)
JSON body	url, type
Success	**200** Recording added
Errors	**400** Invalid ID · The link must start with http:// or https:// · url is required **403** Unauthorized **404** Candidate not found

POST `/api/admin/ats/candidates/{candidate_id}/send-status-email`

Send status email

Access	Logged in; Recruitment access (admin/owner, HR department, or Special Access “ats”) — edit
Path	candidate_id (ID)
JSON body	status
Success	**200** Status email for '…' queued
Errors	**400** Invalid ID · No email template for '…'. Available: … · status is required **403** Unauthorized **404** Candidate not found

PUT `/api/admin/ats/candidates/{candidate_id}/status`

Update status

Access	Logged in; Recruitment access (admin/owner, HR department, or Special Access “ats”) — edit
Path	candidate_id (ID)
JSON body	status
Success	**200** JSON
Errors	**400** Invalid ID · status must be one of: … **403** Unauthorized **404** Candidate not found

POST `/api/admin/ats/candidates/upload`

Store a resume and return suggestions for the Add Candidate form.

Access	Logged in; Recruitment access (admin/owner, HR department, or Special Access “ats”) — edit
Rate limit	20 per minute
Files	file (multipart/form-data)
Success	**200** JSON with resume_url, parsed, parse_status, ai_used, duplicate
Errors	**400** error message (varies with the cause) · Please upload a PDF or Word (.docx) resume. · That file is too large (max 10 MB). · file is required **403** Unauthorized **500** Upload failed: …

GET `/api/admin/ats/stats`

Stats

Access	Logged in; Recruitment access (admin/owner, HR department, or Special Access “ats”); managers see only their own departments
Success	**200** JSON with total, offers_released, offers_accepted, hired, joining_ratio, by_status, by_source, by_department, by_role, time_to_hire, source_effectiveness
Errors	**403** Unauthorized

GET `/api/ats/documents/{doc_token}`

Get doc checklist

Access	Public — protected by the candidate's private document-upload link token
Path	doc_token (secret link token)
Success	**200** JSON with candidate_name, job_role, required, optional_names, documents
Errors	**404** Invalid or expired link

POST `/api/ats/documents/{doc_token}`

Upload doc

Access	Public — protected by the candidate's private document-upload link token
Path	doc_token (secret link token)
Form fields	doc_name
Files	document (multipart/form-data)
Success	**200** JSON with url, documents
Errors	**400** File too large (max 15 MB) · Only PDF and image files (JPEG, PNG) are accepted · doc_name and document are required **404** Invalid or expired link **500** Upload failed: …

3.15 Jobs & Employee Referrals

Job postings (with salary and referral visibility switches) and employee referrals.

Method	Path	Summary
GET	/api/admin/career/jobs	List jobs
POST	/api/admin/career/jobs	Create job
PUT	/api/admin/career/jobs/{job_id}	Update job
DELETE	/api/admin/career/jobs/{job_id}	Delete job
GET	/api/admin/career/referrals	List referrals
PUT	/api/admin/career/referrals/{referral_id}	Update referral
GET	/api/career/jobs	Open job listings — no authentication required.
POST	/api/career/referrals	Submit referral
GET	/api/my/referrals	My referrals

GET `/api/admin/career/jobs`

List jobs

Access	Logged in; Admin / Owner; Special Access: “career”
Success	**200** JSON
Errors	**403** Unauthorized

POST `/api/admin/career/jobs`

Create job

Access	Logged in; Admin / Owner; Special Access: “career” (edit)
JSON body	requirements, salary_min, salary_max, salary_visible, referral_open, title, department, location, description, status, employment_type
Success	**201** JSON
Errors	**400** status must be 'active' or 'closed' · title is required **403** Unauthorized

PUT `/api/admin/career/jobs/{job_id}`

Update job

Access	Logged in; Admin / Owner; Special Access: “career” (edit)
Path	job_id (ID)
JSON body	title, department, location, employment_type, description, salary_min, salary_max, salary_visible, referral_open, requirements, status
Success	**200** Job updated
Errors	**400** Invalid ID · status must be 'active' or 'closed' · title cannot be blank **403** Unauthorized **404** Job not found

DELETE `/api/admin/career/jobs/{job_id}`

Delete job

Access	Logged in; Admin / Owner; Special Access: “career” (edit)
Path	job_id (ID)
Success	**200** Job deleted
Errors	**400** Invalid ID **403** Unauthorized **404** Job not found

GET `/api/admin/career/referrals`

List referrals

Access	Logged in; Admin / Owner; Special Access: “ats”, “career”
Query	job_id
Success	**200** JSON
Errors	**403** Unauthorized

PUT `/api/admin/career/referrals/{referral_id}`

Update referral

Access	Logged in; Admin / Owner; Special Access: “career” (edit)
Path	referral_id (ID)
JSON body	status, candidate_id
Success	**200** Referral status updated
Errors	**400** Invalid ID · status must be one of: … **403** Unauthorized **404** Referral not found

GET `/api/career/jobs`

Open job listings — no authentication required.

Access	Public (no login required)
Query	referral
Success	**200** JSON

POST `/api/career/referrals`

Submit referral

Access	Logged in; role check: employee / manager
Form fields	job_id, candidate_name, candidate_email, candidate_phone, resume_url, notes
Files	resume (multipart/form-data)
Success	**201** JSON
Errors	**400** File is not a valid PDF. · File too large (max 5 MB). · Invalid file type. · Invalid job ID · Only PDF files are allowed. · The resume link must start with http:// or https:// · This job is not open for referrals · candidate_name and candidate_email are required **403** Unauthorized **404** Job not found or no longer open **500** Resume upload failed. Please try again.

GET `/api/my/referrals`

My referrals

Access	Any logged-in user (own records only)
Success	**200** JSON

3.16 Assessments

Create online tests, invite candidates/employees, and the public token-link pages candidates use to take them.

Method	Path	Summary
GET	/api/admin/assessments	List assessments
POST	/api/admin/assessments	Create assessment
PUT	/api/admin/assessments/{assessment_id}	Update assessment
DELETE	/api/admin/assessments/{assessment_id}	Delete assessment
GET	/api/admin/assessments/candidates	List candidates
GET	/api/admin/assessments/candidates/{candidate_id}/result	Candidate result
POST	/api/admin/assessments/invite	Invite candidate
GET	/api/assessment/{token}	Safe-to-call-repeatedly preview: title/description/duration/question COUNT only — never th
POST	/api/assessment/{token}/start	The one-shot "Begin Assessment" action — the only place `status` transitions to "started"
POST	/api/assessment/submit	Submit assessment

GET `/api/admin/assessments`

List assessments

Access	Logged in; Admin / Owner; Special Access: “assessment”
Success	**200** JSON
Errors	**403** Unauthorized

POST `/api/admin/assessments`

Create assessment

Access	Logged in; Admin / Owner; Special Access: “assessment” (edit)
JSON body	questions, duration, passing_score, title, duration_minutes, description
Success	**201** JSON
Errors	**400** title is required **403** Unauthorized

PUT `/api/admin/assessments/{assessment_id}`

Update assessment

Access	Logged in; Admin / Owner; Special Access: “assessment” (edit)
Path	assessment_id (ID)
JSON body	title, description, questions, passing_score, duration, duration_minutes
Success	**200** Assessment updated
Errors	**400** Invalid ID **403** Unauthorized **404** Assessment not found

DELETE `/api/admin/assessments/{assessment_id}`

Delete assessment

Access	Logged in; Admin / Owner; Special Access: “assessment” (edit)
Path	assessment_id (ID)
Success	**200** Assessment deleted
Errors	**400** Invalid ID **403** Unauthorized **404** Assessment not found

GET `/api/admin/assessments/candidates`

List candidates

Access	Logged in; Admin / Owner; Special Access: “assessment”
Query	assessmentId
Success	**200** JSON
Errors	**403** Unauthorized

GET `/api/admin/assessments/candidates/{candidate_id}/result`

Candidate result

Access	Logged in; Admin / Owner; Special Access: “assessment”
Path	candidate_id (ID)
Success	**200** JSON
Errors	**400** Invalid ID **403** Unauthorized **404** Candidate not found

POST `/api/admin/assessments/invite`

Invite candidate

Access	Logged in; Admin / Owner; Special Access: “assessment” (edit)
JSON body	name, email, phone, assessmentId
Success	**201** JSON
Errors	**400** name, email, and assessmentId are required **403** Unauthorized **404** Assessment not found

GET `/api/assessment/{token}`

Safe-to-call-repeatedly preview: title/description/duration/question COUNT only — never the actual question text/options, and never mutates `status`.

Access	Public — protected by the candidate's private assessment link token
Path	token (secret link token)
Success	**200** JSON with title, description, duration, passing_score, question_count
Errors	**404** Assessment not found. · Link is invalid or has already been used. · This assessment has already been submitted. · This assessment was already started and can only be attempted once. Contact HR if you were disconnected.

POST `/api/assessment/{token}/start`

The one-shot "Begin Assessment" action — the only place `status` transitions to "started" and the only place the actual question text/options are ever transmitted.

Access	Public — protected by the candidate's private assessment link token
Path	token (secret link token)
Success	**200** JSON with title, description, duration, passing_score, questions
Errors	**404** Assessment not found. · Link is invalid or has already been used. · This assessment has already been submitted. · This assessment was already started and can only be attempted once. Contact HR if you were disconnected. **409** This assessment was already started and can only be attempted once. Contact HR if you were disconnected.

POST `/api/assessment/submit`

Submit assessment

Access	Public — protected by the candidate's private assessment link token
JSON body	token, answers, forced, timed_out
Success	**200** JSON with score, passed
Errors	**400** Invalid submission token. · This assessment has already been submitted. · answers must be a list. · token is required **404** Assessment not found.

3.17 Learning (LMS)

Courses, modules and lessons, assignments, progress and completion for admins, managers and employees.

Method	Path	Summary
GET	/api/admin/lms/courses	List courses
POST	/api/admin/lms/courses	Create course
PUT	/api/admin/lms/courses/{course_id}	Update course
DELETE	/api/admin/lms/courses/{course_id}	Delete course
POST	/api/admin/lms/courses/{course_id}/assign	Assign course
GET	/api/admin/lms/progress	Progress
PUT	/api/admin/lms/progress/{progress_id}	Update assignment
PATCH	/api/admin/lms/progress/{progress_id}	Update assignment
DELETE	/api/admin/lms/progress/{progress_id}	Delete assignment
GET	/api/manager/lms/courses	List courses
POST	/api/manager/lms/courses	Create course
PUT	/api/manager/lms/courses/{course_id}	Update course
DELETE	/api/manager/lms/courses/{course_id}	Delete course
POST	/api/manager/lms/courses/{course_id}/assign	Assign course
GET	/api/manager/lms/progress	Lms progress
PUT	/api/manager/lms/progress/{progress_id}	Update assignment
PATCH	/api/manager/lms/progress/{progress_id}	Update assignment
DELETE	/api/manager/lms/progress/{progress_id}	Delete assignment
GET	/api/my/lms/courses	My lms courses
POST	/api/my/lms/lessons/{lesson_id}/complete	Complete lesson

GET `/api/admin/lms/courses`

List courses

Access	Logged in; Admin / Owner or Special Access “lms”
Success	**200** JSON

POST `/api/admin/lms/courses`

Create course

Access	Logged in; Admin / Owner or Special Access “lms” — edit
JSON body	expiry_date, tags, modules, title, description, category, thumbnail_url, content_url, course_code, created_by_name, department
Success	**201** JSON
Errors	**400** title is required

PUT `/api/admin/lms/courses/{course_id}`

Update course

Access	Logged in; Admin / Owner or Special Access “lms” — edit
Path	course_id (ID)
JSON body	title, description, category, thumbnail_url, content_url, tags, course_code, created_by_name, department, modules, expiry_date
Success	**200** Course updated
Errors	**400** Invalid ID **404** Course not found

DELETE `/api/admin/lms/courses/{course_id}`

Delete course

Access	Logged in; Admin / Owner or Special Access “lms” — edit
Path	course_id (ID)
Success	**200** Course deleted
Errors	**400** Invalid ID **404** Course not found

POST `/api/admin/lms/courses/{course_id}/assign`

Assign course

Access	Logged in; Admin / Owner or Special Access “lms” — edit
Path	course_id (ID)
JSON body	scheduled_at, department, due_date, employee_ids, departments
Success	**200** JSON with assigned, skipped
Errors	**400** Invalid course ID · No employees to assign **404** Course not found

GET `/api/admin/lms/progress`

Progress

Access	Logged in; Admin / Owner or Special Access “lms”
Query	course_id
Success	**200** JSON

PUT `/api/admin/lms/progress/{progress_id}`

Update assignment

Access	Logged in; Admin / Owner or Special Access “lms” — edit
Path	progress_id (ID)
Success	**200** JSON

PATCH `/api/admin/lms/progress/{progress_id}`

Update assignment

Access	Logged in; Admin / Owner or Special Access “lms” — edit
Path	progress_id (ID)
Success	**200** JSON

DELETE `/api/admin/lms/progress/{progress_id}`

Delete assignment

Access	Logged in; Admin / Owner or Special Access “lms” — edit
Path	progress_id (ID)
Success	**200** Assignment removed
Errors	**400** Invalid assignment ID **404** Assignment not found

GET `/api/manager/lms/courses`

List courses

Access	Logged in; role check: manager
Success	**200** JSON
Errors	**403** Unauthorized

POST `/api/manager/lms/courses`

Create course

Access	Logged in; role check: manager
JSON body	expiry_date, modules, title, description, category, thumbnail_url, course_code, created_by_name, department
Success	**201** JSON
Errors	**400** title is required **403** Unauthorized

PUT `/api/manager/lms/courses/{course_id}`

Update course

Access	Logged in; role check: manager
Path	course_id (ID)
JSON body	title, description, category, thumbnail_url, course_code, created_by_name, department, modules, expiry_date
Success	**200** Course updated
Errors	**400** Invalid ID **403** Unauthorized · You can only edit courses you created **404** Course not found

DELETE `/api/manager/lms/courses/{course_id}`

Delete course

Access	Logged in; role check: manager
Path	course_id (ID)
Success	**200** Course deleted
Errors	**400** Invalid ID **403** Unauthorized · You can only delete courses you created **404** Course not found

POST `/api/manager/lms/courses/{course_id}/assign`

Assign course

Access	Logged in; role check: admin / manager; managers see only their own departments
Path	course_id (ID)
JSON body	scheduled_at, due_date, employee_ids
Success	**200** JSON with assigned, skipped
Errors	**400** Employees not in your department: … · Invalid course ID · employee_ids is required **403** Access denied · Unauthorized **404** Course not found

GET `/api/manager/lms/progress`

Lms progress

Access	Logged in; role check: manager; managers see only their own departments
Query	course_id
Success	**200** JSON
Errors	**403** Unauthorized

PUT `/api/manager/lms/progress/{progress_id}`

Update assignment

Access	Logged in; role check: manager
Path	progress_id (ID)
Success	**200** JSON
Errors	**403** That assignment isn't for someone on your team. · Unauthorized

PATCH `/api/manager/lms/progress/{progress_id}`

Update assignment

Access	Logged in; role check: manager
Path	progress_id (ID)
Success	**200** JSON
Errors	**403** That assignment isn't for someone on your team. · Unauthorized

DELETE `/api/manager/lms/progress/{progress_id}`

Delete assignment

Access	Logged in; role check: manager
Path	progress_id (ID)
Success	**200** Assignment removed
Errors	**403** That assignment isn't for someone on your team. · Unauthorized

GET `/api/my/lms/courses`

My lms courses

Access	Logged in; managers see only their own departments
Success	**200** JSON list

POST `/api/my/lms/lessons/{lesson_id}/complete`

Complete lesson

Access	Any logged-in user (own records only)
Path	lesson_id (ID)
JSON body	module_index, lesson_index, course_id
Success	**200** JSON with progress_pct, status — “Lesson marked complete”
Errors	**404** Lesson not found

3.18 Daily Work Plans

Employees' daily plans and the team/analytics views built from them.

Method	Path	Summary
GET	/api/admin/work-analytics	Work analytics
GET	/api/admin/work-plans	Work plans
POST	/api/admin/work-plans/{plan_id}/comment	Comment work plan
GET	/api/my/work-analytics	My work analytics
GET	/api/my/work-analytics-ai	My work analytics ai
GET	/api/my/work-plan	Get my work plan
POST	/api/my/work-plan	Upsert my work plan
PUT	/api/my/work-plan/{plan_id}/task/{task_id}	Update my task
POST	/api/my/work-plan/share	Share work plan
GET	/api/my/work-plans	My work plans history

GET `/api/admin/work-analytics`

Work analytics

Access	Logged in; Special Access: “work-by-team”; role check: admin / manager / owner; managers see only their own departments
Query	range
Success	**200** JSON
Errors	**403** Unauthorized

GET `/api/admin/work-plans`

Work plans

Access	Logged in; Special Access: “work-by-team”; role check: admin / manager / owner; managers see only their own departments
Query	date
Success	**200** JSON
Errors	**403** Unauthorized

POST `/api/admin/work-plans/{plan_id}/comment`

Comment work plan

Access	Logged in; Special Access: “work-by-team”; role check: admin / manager / owner; managers see only their own departments
Path	plan_id (ID)
JSON body	comment
Success	**200** Comment saved
Errors	**400** Invalid plan ID **403** Unauthorized **404** Plan not found

GET `/api/my/work-analytics`

My work analytics

Access	Any logged-in user (own records only)
Query	range
Success	**200** JSON

GET `/api/my/work-analytics-ai`

My work analytics ai

Access	Any logged-in user (own records only)
Query	range
Success	**200** JSON with summary
Errors	**502** JSON with error — “AI analytics temporarily unavailable.” **503** AI analytics not configured.

GET `/api/my/work-plan`

Get my work plan

Access	Any logged-in user (own records only)
Query	date
Success	**200** JSON

POST `/api/my/work-plan`

Upsert my work plan

Access	Any logged-in user (own records only)
JSON body	status, notify, tasks, date
Success	**200** JSON
Errors	**400** Invalid date. Use YYYY-MM-DD. · status must be 'draft' or 'submitted' · tasks must be a list

PUT `/api/my/work-plan/{plan_id}/task/{task_id}`

Update my task

Access	Any logged-in user (own records only)
Path	plan_id (ID), task_id (ID)
JSON body	status
Success	**200** JSON
Errors	**400** Invalid plan ID · status is required · status must be one of: … **404** Plan not found or not yours · Task not found in plan

POST `/api/my/work-plan/share`

Share work plan

Access	Any logged-in user (own records only)
JSON body	date
Success	**200** Work plan shared successfully
Errors	**404** No work plan found for that date

GET `/api/my/work-plans`

My work plans history

Access	Any logged-in user (own records only)
Query	range
Success	**200** JSON

3.19 Clients & Client Drive

Client records, shared folders/files and manual updates.

Method	Path	Summary
POST	/api/admin/clients	Create client
PUT	/api/admin/clients/{client_id}	Update client
DELETE	/api/admin/clients/{client_id}	Delete client
GET	/api/clients	List clients
GET	/api/clients/{client_id}	Get client
GET	/api/clients/{client_id}/drive	List drive
DELETE	/api/clients/{client_id}/drive/{item_id}	Delete drive item
POST	/api/clients/{client_id}/drive/files	Upload client file
POST	/api/clients/{client_id}/drive/folders	Create folder
GET	/api/clients/{client_id}/updates	List updates
POST	/api/clients/{client_id}/updates	Post update
DELETE	/api/clients/{client_id}/updates/{update_id}	Delete update

POST `/api/admin/clients`

Create client

Access	Logged in; Admin / Owner; Special Access: “clients” (edit); managers see only their own departments
JSON body	departments, name, description
Success	**201** JSON with _id, name, departments — “Client created”
Errors	**400** name is required **403** Unauthorized **409** A client with that name already exists

PUT `/api/admin/clients/{client_id}`

Update client

Access	Logged in; Admin / Owner; Special Access: “clients” (edit); managers see only their own departments; Admin / Owner, Special Access “clients” (edit), or a manager of one of the client's departments — edit
Path	client_id (ID)
JSON body	name, description, departments
Success	**200** JSON with _id, name, description, departments
Errors	**400** Invalid ID · Nothing to update · You must keep this client in at least one of your own departments. · name cannot be empty **403** Unauthorized **404** Client not found **409** A client with that name already exists

DELETE `/api/admin/clients/{client_id}`

Delete client

Access	Logged in; Admin / Owner, Special Access “clients” (edit), or a manager of one of the client's departments — edit
Path	client_id (ID)
Success	**200** Client deleted
Errors	**400** Invalid ID **403** Unauthorized **404** Client not found

GET `/api/clients`

List clients

Access	Logged in; managers see only their own departments; Admin / Owner, Special Access “clients” (edit), or a manager of one of the client's departments — edit
Success	**200** JSON

GET `/api/clients/{client_id}`

Get client

Access	Logged in; Clients of the caller's departments (admin/owner and any “clients” grant see all); Admin / Owner, Special Access “clients” (edit), or a manager of one of the client's departments — edit
Path	client_id (ID)
Success	**200** JSON with _id, name, description, departments, task_count, can_write
Errors	**400** Invalid ID **403** Unauthorized **404** Client not found

GET `/api/clients/{client_id}/drive`

List drive

Access	Any logged-in user
Path	client_id (ID)
Query	parent_id
Success	**200** JSON with items, breadcrumb, parent_id

DELETE `/api/clients/{client_id}/drive/{item_id}`

Delete drive item

Access	Any logged-in user
Path	client_id (ID), item_id (ID)
Success	**200** JSON with removed — “Deleted”
Errors	**400** Invalid item ID **404** Item not found

POST `/api/clients/{client_id}/drive/files`

Upload client file

Access	Any logged-in user
Path	client_id (ID)
Form fields	parent_id, name
Files	file (multipart/form-data)
Success	**201** JSON with _id, type, name, url, size, mime_type, parent_id
Errors	**400** error message (varies with the cause) · File too large (max 15 MB) · Invalid parent folder · file is required **500** Upload failed: …

POST `/api/clients/{client_id}/drive/folders`

Create folder

Access	Any logged-in user
Path	client_id (ID)
JSON body	parent_id, name
Success	**201** JSON with _id, type, name, parent_id
Errors	**400** Invalid parent folder · name is required

GET `/api/clients/{client_id}/updates`

List updates

Access	Any logged-in user
Path	client_id (ID)
Query	limit
Success	**200** JSON

POST `/api/clients/{client_id}/updates`

Post update

Access	Any logged-in user
Path	client_id (ID)
JSON body	text
Success	**201** JSON with kind, _id, text, posted_by, posted_by_name, at
Errors	**400** text is required

DELETE `/api/clients/{client_id}/updates/{update_id}`

Delete update

Access	Logged in; Admin / Owner; Special Access: “clients” (edit)
Path	client_id (ID), update_id (ID)
Success	**200** Update removed
Errors	**400** Invalid ID **403** Unauthorized **404** Update not found

3.20 Team Chat

Direct messages, channels, unread counts, presence (heartbeat) and typing indicators. The web app polls these every few seconds.

Method	Path	Summary
POST	/api/chat/channels	Create channel
GET	/api/chat/conversations	Current user's DMs + channels, sorted newest-first, with per-conversation unread count.
DELETE	/api/chat/conversations/{conv_id}	Delete conversation
POST	/api/chat/conversations/{conv_id}/clear	Clear for me
POST	/api/chat/conversations/{conv_id}/leave	Leave channel
GET	/api/chat/conversations/{conv_id}/messages	Get messages
POST	/api/chat/conversations/{conv_id}/messages	Send message
DELETE	/api/chat/conversations/{conv_id}/messages	Clear messages
PUT	/api/chat/conversations/{conv_id}/messages/{msg_id}	Edit message
DELETE	/api/chat/conversations/{conv_id}/messages/{msg_id}	Delete message
POST	/api/chat/conversations/{conv_id}/messages/{msg_id}/hide	Hide message
POST	/api/chat/conversations/{conv_id}/read	Mark read

Method	Path	Summary
GET	/api/chat/conversations/{conv_id}/typing	Typing get
POST	/api/chat/conversations/{conv_id}/typing	Typing post
POST	/api/chat/dm	Find-or-create the 1-to-1 DM between the caller and another user (idempotent).
POST	/api/chat/heartbeat	Heartbeat
GET	/api/chat/online	Online
GET	/api/chat/unread	Unread count
GET	/api/chat/users	All staff the caller may message, excluding themselves and anyone off-boarded — an ex-empl

POST `/api/chat/channels`

Create channel

Access	Any logged-in user
JSON body	members, name
Success	**201** JSON with conversation
Errors	**400** name is required

GET `/api/chat/conversations`

Current user's DMs + channels, sorted newest-first, with per-conversation unread count.

Access	Any logged-in user
Success	**200** JSON with conversations

DELETE `/api/chat/conversations/{conv_id}`

Delete conversation

Access	Logged in; Admin / Owner
Path	conv_id (ID)
Success	**200** deleted
Errors	**400** Invalid conversation ID · Only channels can be deleted. DMs cannot be removed. **403** Only the channel creator or an admin may delete this channel **404** Conversation not found

POST `/api/chat/conversations/{conv_id}/clear`

Clear for me

Access	Any logged-in user
Path	conv_id (ID)
Success	**200** JSON with ok
Errors	**404** Conversation not found or access denied

POST `/api/chat/conversations/{conv_id}/leave`

Leave channel

Access	Any logged-in user
Path	conv_id (ID)
Success	**200** JSON with ok
Errors	**400** You cannot leave a DM conversation **404** Conversation not found or access denied

GET `/api/chat/conversations/{conv_id}/messages`

Get messages

Access	Any logged-in user
Path	conv_id (ID)
Success	**200** JSON with messages
Errors	**404** Conversation not found or access denied

POST `/api/chat/conversations/{conv_id}/messages`

Send message

Access	Any logged-in user
Path	conv_id (ID)
JSON body	text
Success	**201** JSON
Errors	**400** text is required **404** Conversation not found or access denied

DELETE `/api/chat/conversations/{conv_id}/messages`

Clear messages

Access	Any logged-in user
Path	conv_id (ID)
Success	**200** cleared
Errors	**404** Conversation not found or access denied

PUT `/api/chat/conversations/{conv_id}/messages/{msg_id}`

Edit message

Access	Any logged-in user
Path	conv_id (ID), msg_id (ID)
JSON body	text
Success	**200** JSON
Errors	**400** Invalid message ID · text is required **403** You can only edit your own messages **404** Conversation not found or access denied · Message not found

DELETE `/api/chat/conversations/{conv_id}/messages/{msg_id}`

Delete message

Access	Logged in; Admin / Owner
Path	conv_id (ID), msg_id (ID)
Success	**200** deleted
Errors	**400** Invalid message ID **403** You can only delete your own messages **404** Conversation not found or access denied · Message not found

POST `/api/chat/conversations/{conv_id}/messages/{msg_id}/hide`

Hide message

Access	Any logged-in user
Path	conv_id (ID), msg_id (ID)
Success	**200** JSON with ok
Errors	**400** Invalid message ID **404** Conversation not found or access denied · Message not found

POST `/api/chat/conversations/{conv_id}/read`

Mark read

Access	Any logged-in user
Path	conv_id (ID)
Success	**200** Marked as read
Errors	**404** Conversation not found or access denied

GET `/api/chat/conversations/{conv_id}/typing`

Typing get

Access	Any logged-in user
Path	conv_id (ID)
Success	**200** JSON with typing
Errors	**404** Conversation not found or access denied

POST `/api/chat/conversations/{conv_id}/typing`

Typing post

Access	Any logged-in user
Path	conv_id (ID)
Success	**200** JSON with ok
Errors	**404** Conversation not found or access denied

POST `/api/chat/dm`

Find-or-create the 1-to-1 DM between the caller and another user (idempotent).

Access	Any logged-in user
JSON body	user_id
Success	**200** JSON with conversation **201** JSON with conversation
Errors	**400** Cannot DM yourself · user_id is required **404** User not found

POST `/api/chat/heartbeat`

Heartbeat

Access	Any logged-in user
Success	**200** JSON with ok

GET `/api/chat/online`

Online

Access	Any logged-in user
Success	**200** JSON with online

GET `/api/chat/unread`

Unread count

Access	Any logged-in user
Success	**200** JSON with count

GET `/api/chat/users`

All staff the caller may message, excluding themselves and anyone off-boarded — an ex-employee showing up as a messageable contact isn't useful, and existing DMs with them still work fine without this.

Access	Any logged-in user
Success	**200** JSON with users

3.21 AI Assistant

Text/voice assistant that answers questions using the caller's own permissions.

Method	Path	Summary
POST	/api/assistant/chat	Text-mode Rexor — every role.
POST	/api/assistant/speak	Speak
POST	/api/assistant/voice	Admin/owner/delegated-access voice mode — same orchestrator, tuned for text-to-speech: sho

POST /api/assistant/chat

Text-mode Rexor — every role.

Access	Any logged-in user
Rate limit	20 per minute
JSON body	history, message
Success	**200** JSON with reply
Errors	**400** Message is too long. · message is required. **503** Assistant not configured.

POST /api/assistant/speak

Speak

Access	Any logged-in user
Rate limit	30 per minute
JSON body	voice, text
Errors	**400** Nothing to speak. · text is required. **502** Voice generation failed. **503** Voice not configured.

POST /api/assistant/voice

Admin/owner/delegated-access voice mode — same orchestrator, tuned for text-to-speech: short, plain sentences, a chosen language, no markdown.

Access	Logged in; role check: admin / owner
Rate limit	20 per minute
JSON body	history, message, lang
Success	**200** JSON with reply, lang
Errors	**400** Message is too long. · message is required. **403** Unauthorized **503** Assistant not configured.

3.22 Announcements, My Profile & Uploads

Company announcements (with attachments) and the caller's own profile and profile picture.

Method	Path	Summary
POST	/api/admin/approve-correction	Approve correction
GET	/api/admin/corrections	Corrections
POST	/api/admin/corrections/backfill	One-off repair: every correction request historically carried a bogus attendance_id ("manu
GET	/api/admin/reports/corrections	HR report: EVERY correction request for a month, regardless of approval_target — unlike /a
GET	/api/announcements	Get announcements
POST	/api/announcements	Create announcement
PUT	/api/announcements/{ann_id}	Update announcement
DELETE	/api/announcements/{ann_id}	Delete announcement
POST	/api/attendance/request-correction	Request correction

Method	Path	Summary
POST	/api/manager/approve-correction	Approve correction
GET	/api/manager/corrections	Corrections
GET	/api/my/corrections	My corrections
GET	/api/my/profile	Get my profile
PUT	/api/my/profile	Update my profile
POST	/api/my/profile-picture	Upload my profile picture
DELETE	/api/my/profile-picture	Remove my profile picture

POST `/api/admin/approve-correction`

Approve correction

Access	Logged in; Admin / Owner
JSON body	id, action
Success	**200** JSON with synced — “Correction …”; JSON with synced — “Correction …, but the attendance record failed to sync: …”
Errors	**400** Invalid ID · id and action are required **403** Unauthorized **404** Not found

GET `/api/admin/corrections`

Corrections

Access	Logged in; Admin / Owner
Success	**200** JSON
Errors	**403** Unauthorized

POST `/api/admin/corrections/backfill`

One-off repair: every correction request historically carried a bogus attendance_id ("manual_entry" — a placeholder string sent by both correction forms, not a real ObjectId).

Access	Logged in; Admin / Owner
Success	**200** JSON with fixed, skipped — “Backfilled … correction(s); … already applied or unparseable.”
Errors	**403** Unauthorized

GET `/api/admin/reports/corrections`

HR report: EVERY correction request for a month, regardless of approval_target — unlike /api/admin/corrections (an approval queue that deliberately only shows requests routed to admin), this is a supe

Access	Logged in; Admin / Owner; Special Access: “attendance”, “summary”
Query	month
Success	**200** JSON
Errors	**400** month required **403** Unauthorized

GET `/api/announcements`

Get announcements

Access	Any logged-in user
Success	**200** JSON

POST `/api/announcements`

Create announcement

Access	Logged in; Admin / Owner; Special Access: “announcements” (edit)
JSON body	title, message
Success	**201** Announcement broadcasted
Errors	**403** Unauthorized

PUT `/api/announcements/{ann_id}`

Update announcement

Access	Logged in; Admin / Owner; Special Access: “announcements” (edit)
Path	ann_id (ID)
JSON body	title, message
Success	**200** Announcement successfully updated.
Errors	**400** Title and message are required fields. **403** Unauthorized access. Admins only. **404** Announcement not found in the database.

DELETE `/api/announcements/{ann_id}`

Delete announcement

Access	Logged in; Admin / Owner; Special Access: “announcements” (edit)
Path	ann_id (ID)
Success	**200** Announcement recalled and deleted successfully.
Errors	**403** Unauthorized access. Admins only. **404** Announcement not found or already deleted.

POST `/api/attendance/request-correction`

Request correction

Access	Logged in; role check: admin / manager / owner
JSON body	attendance_id, new_time, reason, employee_name
Success	**201** Correction request dispatched
Errors	**400** Monthly limit of 3 corrections reached

POST `/api/manager/approve-correction`

Approve correction

Access	Logged in; role check: manager
JSON body	id, action
Success	**200** JSON with synced — “Correction …”; JSON with synced — “Correction …, but the attendance record failed to sync: …”
Errors	**400** action must be 'Approved' or 'Rejected' **403** Unauthorized · Unauthorized: employee is not in your team **404** Not found

GET `/api/manager/corrections`

Corrections

Access	Logged in; role check: manager
Success	**200** JSON
Errors	**403** Unauthorized

GET `/api/my/corrections`

My corrections

Access	Any logged-in user (own records only)
Success	**200** JSON

GET `/api/my/profile`

Get my profile

Access	Any logged-in user (own records only)
Success	**200** JSON with birthday, phone, bio, photo_url, offboarded

PUT `/api/my/profile`

Update my profile

Access	Any logged-in user (own records only)
JSON body	birthday, phone, bio
Success	**200** JSON with success
Errors	**400** Invalid birthday format. Use YYYY-MM-DD.

POST `/api/my/profile-picture`

Upload my profile picture

Access	Any logged-in user (own records only)
Files	file (multipart/form-data)
Success	**200** JSON with photo_url
Errors	**400** error message (varies with the cause) · Image too large (max 5 MB) · Please upload a JPEG, PNG, WEBP, or GIF image. · file is required **500** Upload failed: …

DELETE `/api/my/profile-picture`

Remove my profile picture

Access	Any logged-in user (own records only)
Success	**200** Profile picture removed.

3.23 Notifications

Badge counts and today's birthdays.

Method	Path	Summary
GET	/api/notifications/birthdays	Return users whose birthday (month + day) matches today, scoped by role.
GET	/api/notifications/counts	Get notification counts

GET `/api/notifications/birthdays`

Return users whose birthday (month + day) matches today, scoped by role.

Access	Logged in; role check: employee / manager; managers see only their own departments
Success	**200** JSON list

GET `/api/notifications/counts`

Get notification counts

Access	Logged in; role check: admin / manager / owner
Success	**200** JSON

3.24 Dashboard Statistics

Admin dashboard numbers and the end-of-day automatic-absent job.

Method	Path	Summary
GET	/api/admin/attendance-summary	Attendance summary
GET	/api/admin/today-stats	Today stats
POST	/api/attendance/auto-absent	Cron endpoint — marks absent users who didn't punch in.

GET `/api/admin/attendance-summary`

Attendance summary

Access	Logged in; Admin / Owner; Special Access: “attendance”, “summary”
Query	month
Success	**200** JSON
Errors	**400** month required **403** Unauthorized

GET `/api/admin/today-stats`

Today stats

Access	Logged in; Admin / Owner; Special Access: “attendance”, “summary”; role check: employee / manager
Success	**200** JSON with present, leave, not_checked_in, total_workforce, employee_count, manager_count, by_department
Errors	**403** Unauthorized

POST `/api/attendance/auto-absent`

Cron endpoint — marks absent users who didn't punch in.

Access	Scheduled job — requires the secret header X-Cron-Secret
Headers	X-Cron-Secret
Success	**200** Absent auto-marking completed; Skipped — today is a company holiday
Errors	**401** Unauthorized

3.25 Assets

Company assets, assignment and employee asset requests.

Method	Path	Summary
GET	/api/admin/assets	Get assets
PUT	/api/admin/assets/{asset_id}	Update asset
POST	/api/admin/assets/{asset_id}/assign	Assign asset to office admin
GET	/api/assets/my-requests	Get my assets
POST	/api/assets/request	Request asset
GET	/api/manager/assets	Get assets
PUT	/api/manager/assets/{asset_id}	Update asset
POST	/api/manager/assets/{asset_id}/assign	Assign asset

GET `/api/admin/assets`

Get assets

Access	Logged in; Admin / Owner; Special Access: “assets”
Success	**200** JSON
Errors	**403** Unauthorized access. Admins only.

PUT `/api/admin/assets/{asset_id}`

Update asset

Access	Logged in; Admin / Owner; Special Access: “assets” (edit)
Path	asset_id (ID)
JSON body	admin_status
Success	**200** Asset successfully marked as … by Master Admin.
Errors	**400** Invalid admin status provided. **403** Unauthorized action. Admins only. **404** Asset request not found in database.

POST `/api/admin/assets/{asset_id}/assign`

Assign asset to office admin

Access	Logged in; Admin / Owner; Special Access: “assets” (edit)
Path	asset_id (ID)
JSON body	emails, asset
Success	**200** Assignment emails sent successfully.
Errors	**400** At least one recipient email is required. · Invalid asset ID **403** Unauthorized

GET `/api/assets/my-requests`

Get my assets

Access	Any logged-in user
Success	**200** JSON

POST `/api/assets/request`

Request asset

Access	Any logged-in user
JSON body	asset_name, reason, request_type
Success	**201** JSON with id — “Asset requested successfully.”
Errors	**400** Asset name and reason are strictly required.

GET `/api/manager/assets`

Get assets

Access	Logged in; role check: admin / manager / owner; managers see only their own departments
Success	**200** JSON
Errors	**403** Unauthorized access to team assets.

PUT `/api/manager/assets/{asset_id}`

Update asset

Access	Logged in; role check: admin / manager / owner; managers see only their own departments
Path	asset_id (ID)
JSON body	manager_status
Success	**200** Asset successfully marked as … by Manager.
Errors	**400** Invalid manager status provided. **403** Unauthorized action. · Unauthorized: asset belongs to a different department **404** Asset request not found in database.

POST `/api/manager/assets/{asset_id}/assign`

Assign asset

Access	Logged in; role check: admin / manager / owner; managers see only their own departments
Path	asset_id (ID)
JSON body	emails
Success	**200** Assignment emails sent successfully.
Errors	**400** At least one recipient email is required · Invalid asset ID **403** Unauthorized · You can only assign assets for your own department **404** Asset not found

3.26 Achievements

Employee of the month/quarter/year, awards and hike records.

Method	Path	Summary
GET	/api/admin/achievements	Return all achievements, optionally filtered by employee_id.
POST	/api/admin/achievements	Create achievement
DELETE	/api/admin/achievements/{achievement_id}	Delete achievement

GET /api/admin/achievements

Return all achievements, optionally filtered by employee_id.

Access	Logged in; Admin / Owner
Query	employee_id
Success	**200** JSON
Errors	**403** Unauthorized

POST /api/admin/achievements

Create achievement

Access	Logged in; Admin / Owner
JSON body	employee_name, type, description, month, employee_id, title
Success	**201** JSON
Errors	**400** employee_id and title are required **403** Unauthorized

DELETE /api/admin/achievements/{achievement_id}

Delete achievement

Access	Logged in; Admin / Owner
Path	achievement_id (ID)
Success	**200** JSON with success
Errors	**400** Invalid achievement ID **403** Unauthorized **404** Achievement not found

3.27 My Goals (Sticky Notes)

Personal daily/weekly goals with start date, deadline, sharing and login reminders.

Method	Path	Summary
GET	/api/my/goals	List goals
POST	/api/my/goals	Create goal
PUT	/api/my/goals/{goal_id}	Update goal
DELETE	/api/my/goals/{goal_id}	Delete goal
GET	/api/my/goals/people	Colleagues a note can be shared with — id, name, department only.
GET	/api/my/goals/reminders	Started, not-done goals (own or shared with me) due within REMIND_DAYS or overdue — most u

GET /api/my/goals

List goals

Access	Any logged-in user (own records only)
Success	**200** JSON with open, done, shared, shared_done

POST `/api/my/goals`

Create goal

Access	Any logged-in user (own records only)
JSON body	notes, color, start_date, shared_with, title, type, deadline
Success	**201** JSON
Errors	**400** error message (varies with the cause) · You have … open goals — finish or delete some first.

PUT `/api/my/goals/{goal_id}`

Update goal

Access	Any logged-in user (own records only)
Path	goal_id (ID)
JSON body	done, notes, color, start_date, shared_with, title, type, deadline
Success	**200** JSON
Errors	**400** error message (varies with the cause) · Nothing to update. **403** Only the person who created this note can change it. **404** Goal not found.

DELETE `/api/my/goals/{goal_id}`

Delete goal

Access	Any logged-in user (own records only)
Path	goal_id (ID)
Success	**200** Deleted.
Errors	**403** Only the person who created this note can delete it. **404** Goal not found.

GET `/api/my/goals/people`

Colleagues a note can be shared with — id, name, department only.

Access	Any logged-in user (own records only)
Success	**200** JSON

GET `/api/my/goals/reminders`

Started, not-done goals (own or shared with me) due within REMIND_DAYS or overdue — most urgent first.

Access	Any logged-in user (own records only)
Success	**200** JSON

3.28 Experience Certificates (QR Verification)

Generate, reissue and revoke experience certificates and the public QR verification endpoint.

Method	Path	Summary
GET	/api/admin/certificates	List certificates
POST	/api/admin/certificates	Generate certificate
GET	/api/admin/certificates/{cert_id}/audit	Certificate audit
GET	/api/admin/certificates/{cert_id}/pdf	Certificate pdf
POST	/api/admin/certificates/{cert_id}/reissue	Reissue certificate
POST	/api/admin/certificates/{cert_id}/revoke	Revoke certificate
GET	/api/admin/certificates/eligible	Eligible employees
GET	/api/public/certificates/{token}	Always answers 200 with a `status`: verified | revoked | superseded | invalid, so the page

GET `/api/admin/certificates`

List certificates

Access	Logged in; Admin / Owner or Special Access “certificates” (view)
Query	employee_id, status
Success	**200** JSON list
Errors	**403** Unauthorized

POST `/api/admin/certificates`

Generate certificate

Access	Logged in; Admin / Owner or Special Access “certificates” (edit)
JSON body	employee_id, signatory_name, signatory_designation
Success	**201** JSON
Errors	**400** error message (varies with the cause) **403** Unauthorized **404** Employee not found. **409** … already has an active certificate (…). Use Reissue to replace it.

GET `/api/admin/certificates/{cert_id}/audit`

Certificate audit

Access	Logged in; Admin / Owner or Special Access “certificates” (view)
Path	cert_id (ID)
Success	**200** JSON with reference_no, events, verifications_30d, verify_count
Errors	**403** Unauthorized **404** Certificate not found.

GET `/api/admin/certificates/{cert_id}/pdf`

Certificate pdf

Access	Logged in; Admin / Owner or Special Access “certificates” (view)
Path	cert_id (ID)
Success	**200** file download
Errors	**403** Unauthorized **404** Certificate not found.

POST `/api/admin/certificates/{cert_id}/reissue`

Reissue certificate

Access	Logged in; Admin / Owner or Special Access “certificates” (edit)
Path	cert_id (ID)
JSON body	reason, signatory_name, signatory_designation
Success	**201** JSON
Errors	**400** error message (varies with the cause) · Only an active certificate can be reissued (this one is …). · Please give a reason for the reissue. **403** Unauthorized **404** Certificate not found. · The employee record no longer exists.

POST `/api/admin/certificates/{cert_id}/revoke`

Revoke certificate

Access	Logged in; Admin / Owner or Special Access “certificates” (edit)
Path	cert_id (ID)
JSON body	reason
Success	**200** JSON
Errors	**400** A reason is required to revoke a certificate. · Only an active certificate can be revoked (this one is …). **403** Unauthorized **404** Certificate not found.

GET `/api/admin/certificates/eligible`

Eligible employees

Access	Logged in; Admin / Owner or Special Access “certificates” (view)
Success	**200** JSON
Errors	**403** Unauthorized

GET `/api/public/certificates/{token}`

Always answers 200 with a `status`: verified | revoked | superseded | invalid, so the page can render every case.

Access	Public — protected by the unguessable token inside the certificate's QR code
Rate limit	60 per minute
Path	token (secret link token)

Appendix A — Public endpoints (no login)

Method	Path	How it is protected
GET	/	Public (no login required)
GET	/api/health	Public (no login required)
GET	/health	Public (no login required)
GET	/uploads/{path:filename}	Public (no login required)
POST	/api/forgot-password	Public (no login) — part of signing in or recovering a password
POST	/api/login	Public (no login) — part of signing in or recovering a password
POST	/api/reset-password	Public (no login) — part of signing in or recovering a password
POST	/api/login/mfa	Public (no login) — part of signing in or recovering a password
POST	/api/register-admin	Open only while no admin/owner exists (first-time setup); otherwise needs an admin/owner login
GET	/iclock/cdata	Public address, used by biometric devices — the device's registered serial number (?SN=) is the credential
POST	/iclock/cdata	Public address, used by biometric devices — the device's registered serial number (?SN=) is the credential
GET	/iclock/getrequest	Public address, used by biometric devices — the device's registered serial number (?SN=) is the credential
GET	/api/ats/documents/{doc_token}	Public — protected by the candidate's private document-upload link token
POST	/api/ats/documents/{doc_token}	Public — protected by the candidate's private document-upload link token
GET	/api/career/jobs	Public (no login required)
GET	/api/assessment/{token}	Public — protected by the candidate's private assessment link token
POST	/api/assessment/{token}/start	Public — protected by the candidate's private assessment link token
POST	/api/assessment/submit	Public — protected by the candidate's private assessment link token
POST	/api/attendance/auto-absent	Scheduled job — requires the secret header X-Cron-Secret
GET	/api/public/certificates/{token}	Public — protected by the unguessable token inside the certificate's QR code

Appendix B — Rate limits

Method	Path	Limit
POST	/api/forgot-password	5 per hour
POST	/api/login	10 per minute
POST	/api/reset-password	10 per hour
POST	/api/login/mfa	15 per minute
POST	/api/my/mfa/disable	10 per minute
POST	/api/my/mfa/enable	10 per minute
GET	/iclock/cdata	120 per minute
POST	/iclock/cdata	120 per minute
POST	/api/admin/ats/ai-overview	10 per minute
POST	/api/admin/ats/candidates/upload	20 per minute
POST	/api/assistant/chat	20 per minute
POST	/api/assistant/speak	30 per minute
POST	/api/assistant/voice	20 per minute
GET	/api/public/certificates/{token}	60 per minute

All other routes have no per-route limit.

Appendix C — Statistics

Method	Count
DELETE	31
GET	119
PATCH	2
POST	94
PUT	31